# Data Structure
# ——List

Liu Haiyang
Beijing Jiaotong University

# Data Structure [Wikipedia]

- Data Structure is a particular way of storing and organizing data in a computer so that it can be used efficiently.
- Different kinds of data structures are suited to different kinds of applications.
- Storing and retrieving can be carried out on data stored in memory.

# Merriam-Webster's Definition

- Way in which data are stored for efficient search and retrieval.
- The simplest data structure is the one-dimensional (linear) array.
- Data items stored non-consecutively in memory may be linked by pointers.
- Many algorithms have been developed for storing data efficiently

# Algorithms [Wikipedia]

- An algorithm is a step-by-step procedure for calculations.
- An algorithm is an effective method expressed as a finite list of well-defined instructions for calculating a function.
- The transition from one state to the next is not necessarily deterministic; some algorithms incorporate random input.

# Merriam-Webster's Definition

- Procedure that produces the answer to a question or the solution to a problem in a finite number of steps.
- An algorithm that produces a yes or no answer is called a decision procedure; one that leads to a solution is a computation procedure.
- Example: A mathematical formula and the instructions in a computer program

# ADT - General Concept

- Problem solving with a computer means processing data
- To process data, we need to define the data type and the operation to be performed on the data
- The definition of the data type and the definition of the operation to be applied to the data is part of the idea behind an Abstract Data Type (ADT)

# ADT - General Concept

- The user of an ADT needs only to know that a set of operations are available for the data type, but does not need to know how they are applied
- Several simple ADTs, such as integer, real, character, pointer and so on, have been implemented and are available for use in most languages

# Data Types

- A data type is characterized by:
  - A set of *values*
  - A *data representation,* which is common to all these values, and
  - A set of *operations,* which can be applied uniformly to all these values

# Primitive Data Types

- Languages like ‘C’ provides the following primitive data types:
  - boolean
  - char, byte, int
  - float, double
- Each primitive type has:
  - A set of values
  - A data representation
  - A set of operations
- These are “set in stone”.

# ADT Definition [Wikipedia]

- In computer science, an abstract data type (ADT) is a mathematical model for a certain class of data structures that have similar behavior.
- An abstract data type is defined indirectly, only by the operations that may be performed on it and by mathematical constraints on the effects (and possibly cost) of those operations.

# ADT Definition [Wikipedia]

- An ADT may be implemented by specific data types or data structures, in many ways and in many programming languages; or described in a formal specification language.
- example, an abstract stack could be defined by three operations:
  - push, that inserts some data item onto the structure,
  - pop, that extracts an item from it, and
  - peek, that allows data on top of the structure to be examined without removal.

# Definition from techforum4you

- Abstract data types or ADTs are a mathematical specification of a set of data and the set of operations that can be performed on the data.
- They are abstract in the sense that the focus is on the definitions and the various operations with their arguments.
- The actual implementation is not defined, and does not affect the use of the ADT.

# ADT in Simple Words

- Definition:
  - Is a set of operation
  - Mathematical abstraction
  - No implementation detail
- Example:
  - Lists, sets, graphs, stacks are examples of ADT along with their operations

# Why ADT?

- Modularity
  - divide program into small functions
  - easy to debug and maintain
  - easy to modify
  - group work
- Reuse
  - do some operations only once
- Easy to change the implementation
  - transparent to the program

# Implementing an ADT

- To implement an ADT, you need to choose:
  - A data representation
    - must be able to represent all necessary values of the ADT
  - An algorithm for each of the necessary operation:
    - must be consistent with the chosen representation
- Remember: Once other people are using it
  - It's easy to add functionality

# The List ADT

- The List is an
  - Ordered sequence of data items called elements
  - $A_1, A_2, A_3, \cdots, A_N$ is a list of size N
  - size of an empty list is 0
  - $A_{i+1}$ succeeds $A_i$
  - $A_{i-1}$ preceeds $A_i$
  - Position of $A_i$ is i
  - First element is $A_1$ called “head”
  - Last element is $A_N$ called “tail”

# Operations on Lists

- MakeEmpty
- PrintList
- Find
- FindKth
- Insert
- Delete
- Next
- Previous

# List – An Example

- The elements of a list are 34, 12, 52, 16, 12
  - Find (52) -> 3
  - Insert (20, 4) -> 34, 12, 52, 20, 16, 12
  - Delete (52) -> 34, 12, 20, 16, 12
  - FindKth (3) -> 20

# List - Implementation

- Lists can be implemented using:
  - Arrays
  - Linked List

# Arrays

- Array is a static data structure that represents a collection of fixed number of homogeneous data items
- A fixed-size indexed sequence of elements, all of the same type.
- The individual elements are typically stored in consecutive memory locations.
- The length of the array is determined when the array is created, and cannot be changed.

# Arrays

- Any component of the array can be inspected or updated by using its index.
  - This is an efficient operation
  - O(1) = constant time
- The array indices may be integers (C, Java) or other discrete data types (Pascal, Ada).
- The lower bound may be zero (C, Java), one (Fortran), or chosen by the programmer (Pascal, Ada)

# Different Types of Arrays

- One-dimensional array: only one index is used
- Multi-dimensional array: array involving more than one index
- Static array: the compiler determines how memory will be allocated for the array
- Dynamic array: memory allocation takes place during execution

# One Dimensional Static Array

- Syntax:
  - **`ElementType arrayName [CAPACITY];`**
  - **`ElementType arrayName [CAPACITY] = { initializer_list };`**
- Example in C:
  - **`int  b [5];`**
  - **`int  b [5] = {19, 68, 12, 45, 72};`**

# Array Output Function

```
void display(int array[],int num_values)
{
 for (int I = 0; i<num_values; i++)
    printf(array[i]);
}
```

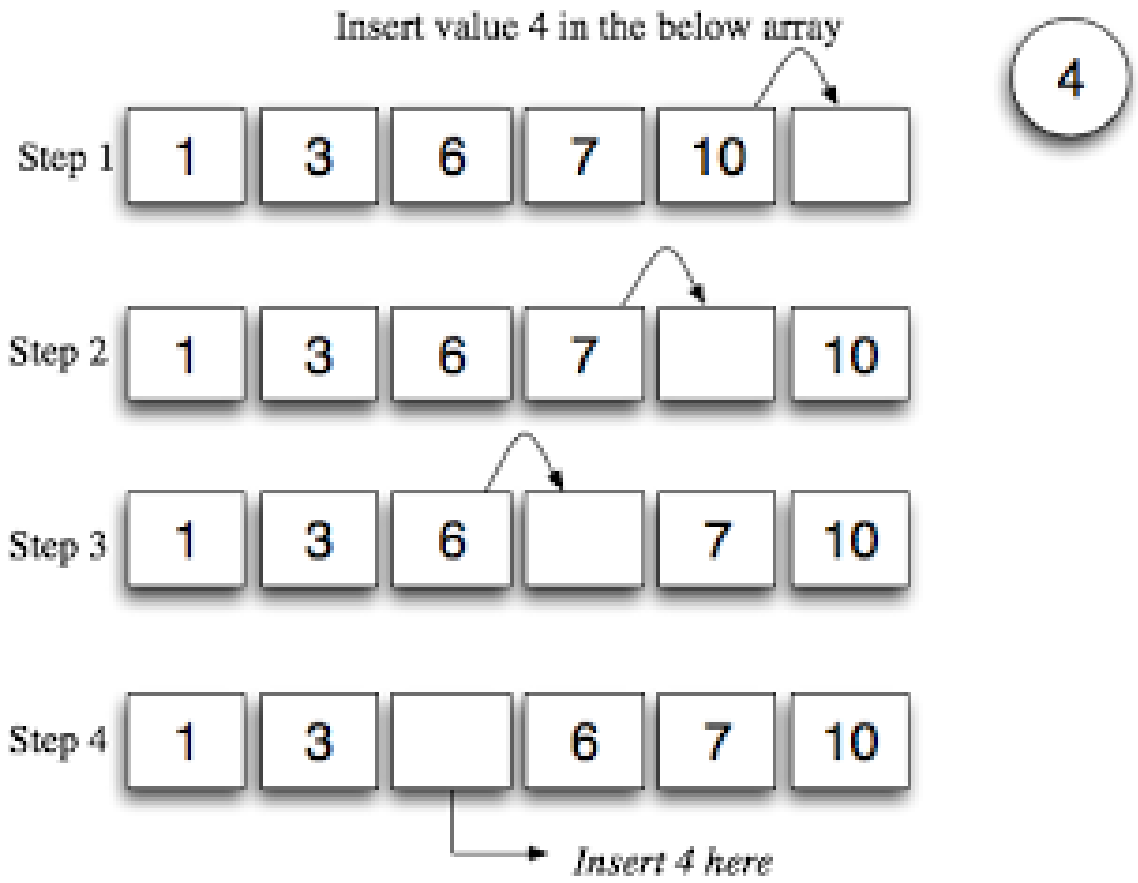
# List Implemented Using Array

# Operations On Lists

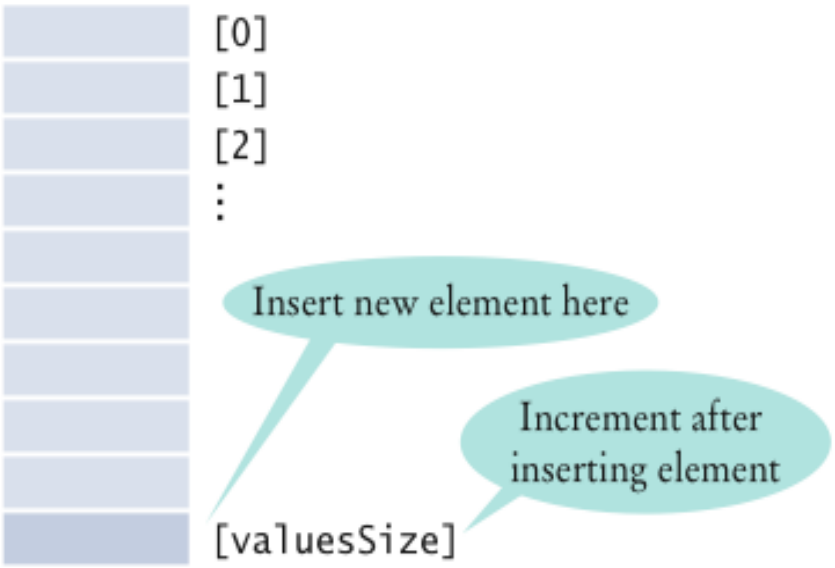
- We'll consider only few operations and not all operations on Lists
- Let us consider Insert
- There are two possibilities:
  - Ordered List
  - Unordered List

# Insertion into an Ordered List

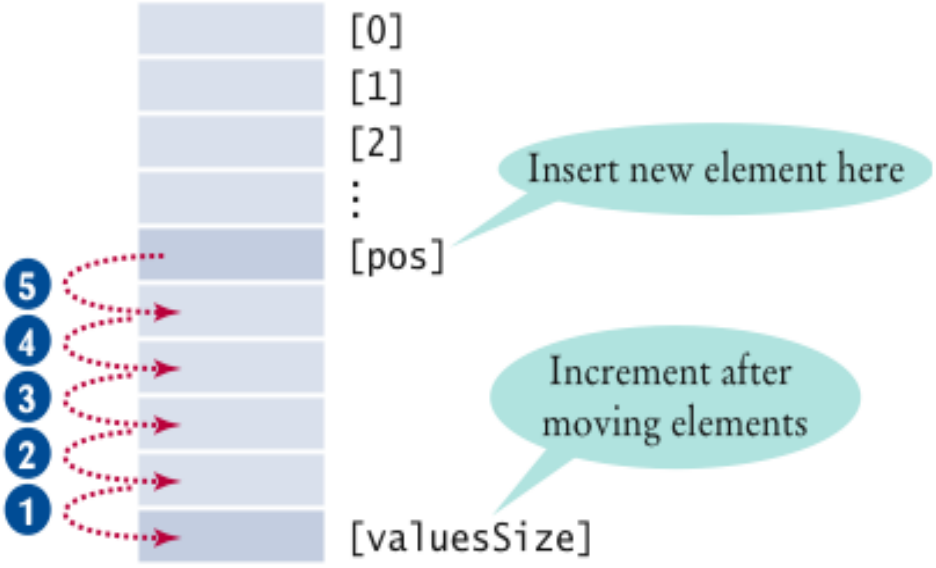
# Insertion in Detail



# Insertion

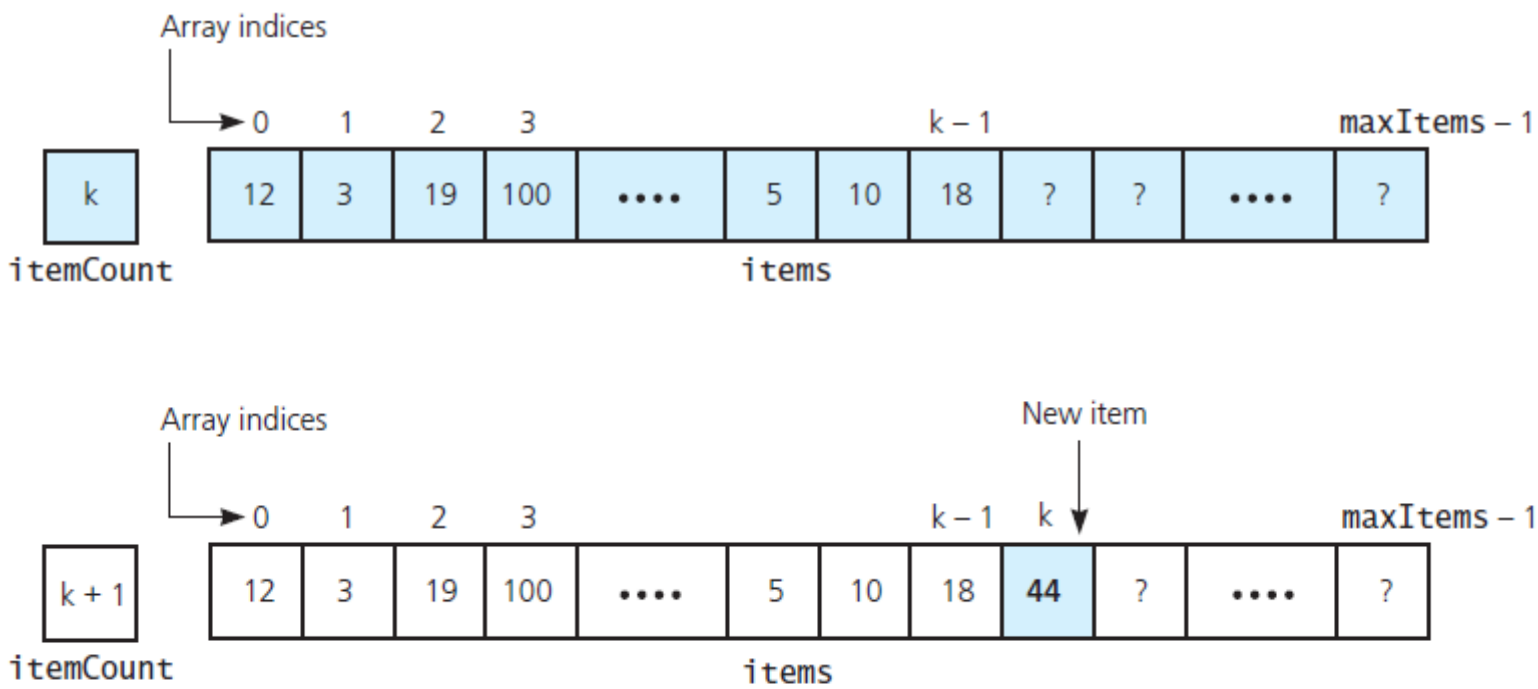



Inserting an Element in an Unordered Array

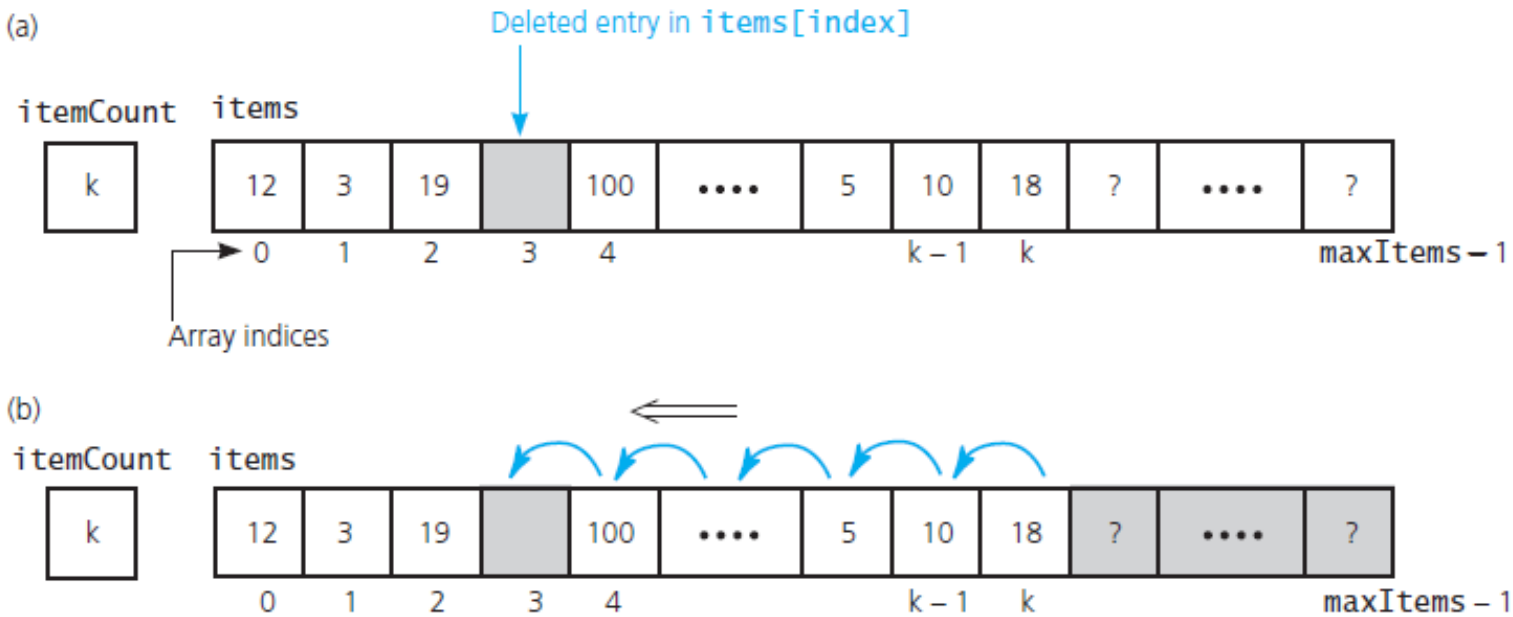



Inserting an Element in an Ordered Array

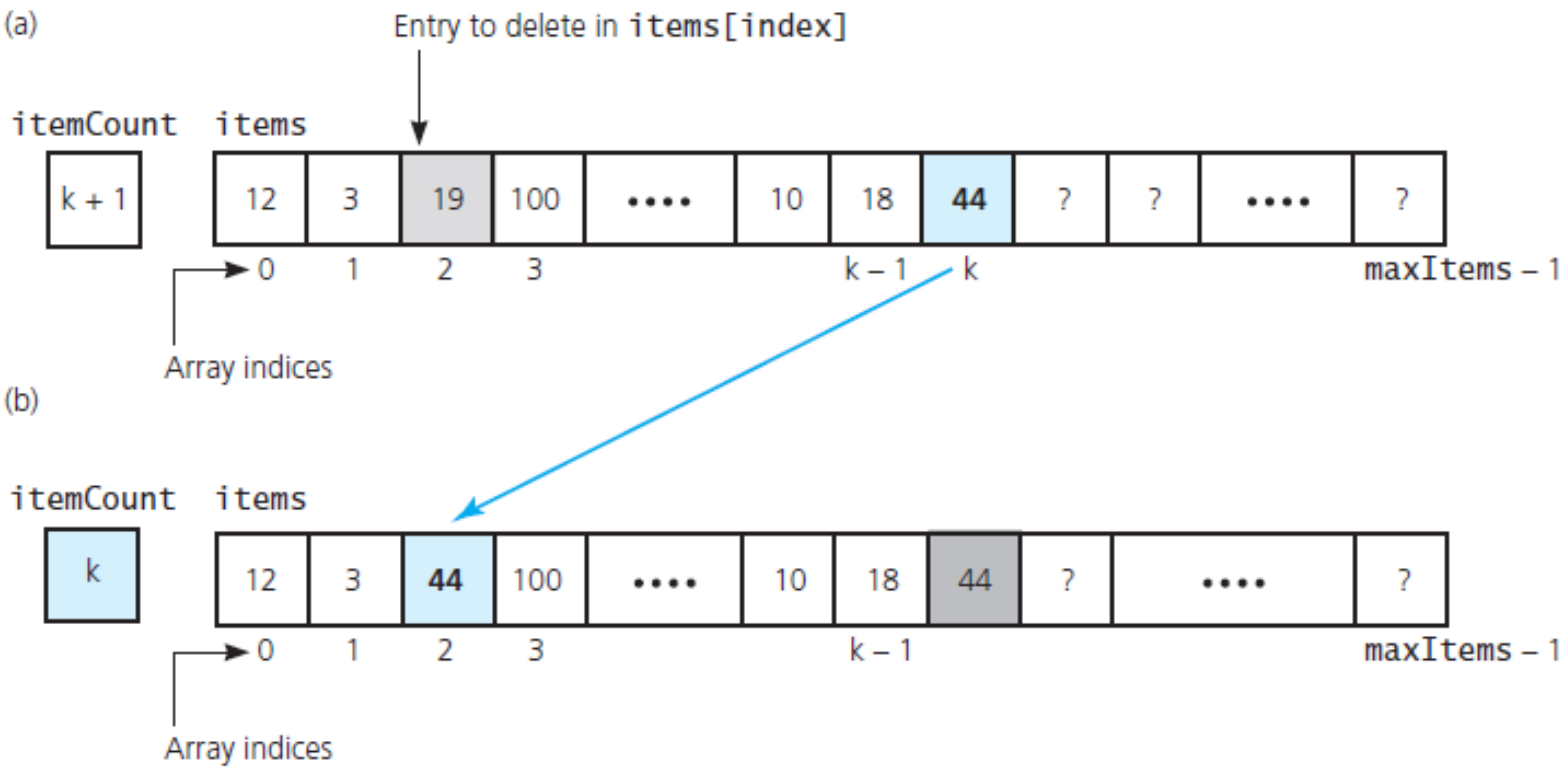
# Insertion into an Unordered List



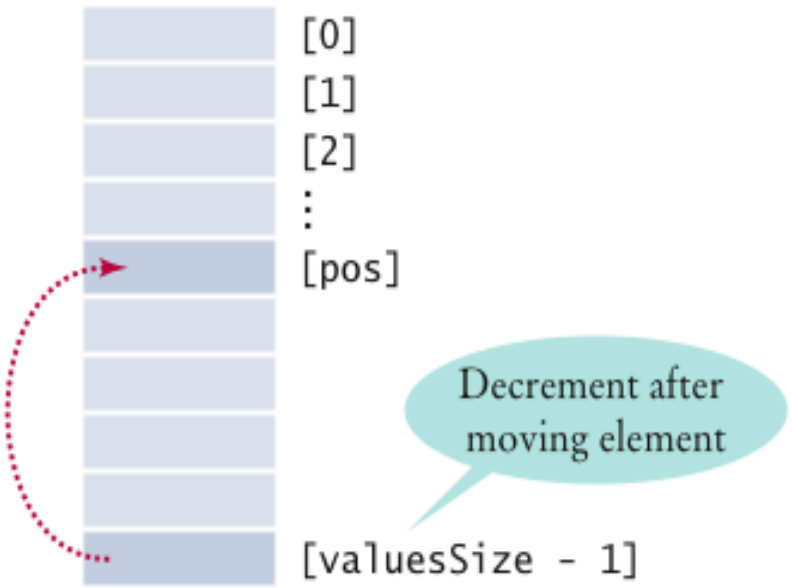
# Implementing Deletion



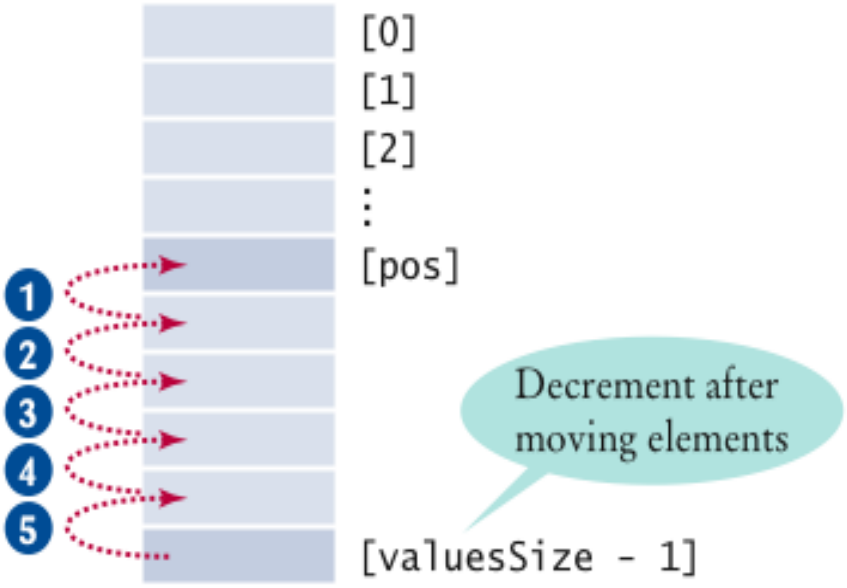
# Deletion - Another Method



# Deletion




Removing an Element in an Unordered Array




Removing an Element in an Ordered Array

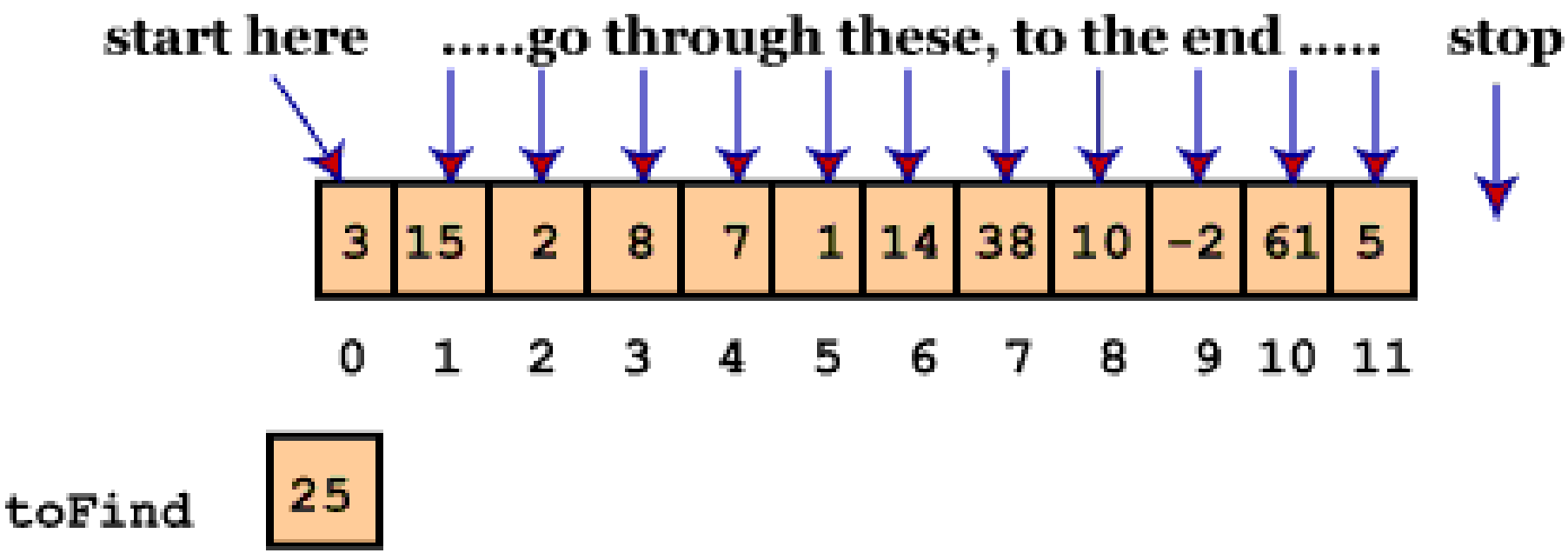
# Find / Search

- Searching is the process of looking for a specific element in an array
- For example, discovering whether a certain score is included in a list of scores.
- Searching, like sorting, is a common task in computer programming.
- There are many algorithms and data structures devoted to searching.
- The most common one is the linear search.

# Linear Search

- The linear search approach compares the given value with each element in the array.
- The method continues to do so until the given value matches an element in the list or the list is exhausted without a match being found.
- If a match is made, the linear search returns the index of the element in the array that matches the key.
- If no match is found, the search returns -1.

# Linear Search



# Linear Search Function

```
int LinearSearch (int a[], int n, int key)
{
   int i;
   for(i=0; i<n; i++)
   {
     if (a[i] == key)
        return i;
   }
   return -1;
}
```

# Using the Function

- **`LinearSearch (array, n, key)`**
- Here "array" is an array of the size n.
- This algorithm finds the location of the element “key" in the array "array".
- If search item is found, it sets loc to the index of the element; otherwise, it sets loc to -1
- **`index=linearsearch(array, n, key)`**

# PrintList Operation

```
int myArray [5] = {19,68,12,45,72};
/* To print all the elements of the array
for (int i=0;i<5;i++)
{
printf("%d", myArray[i]);
}
```

# Operations Running Times

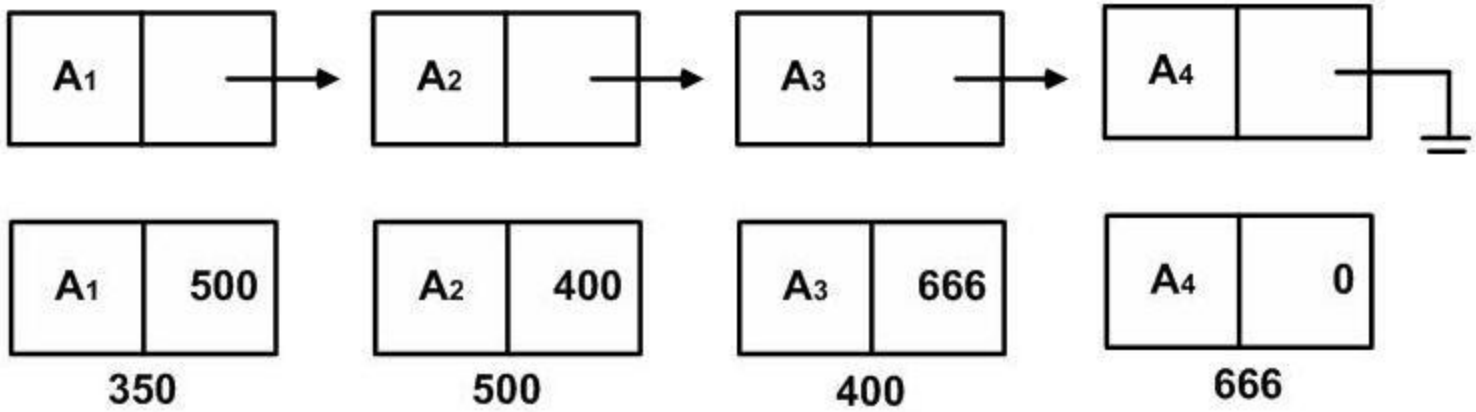
| | |
|---|---|
| PrintList<br>Find | O(N) |
| Insert<br>Delete | O(N) (on avarage half needs to be moved) |
| FindKth | O(1) |

# Disadvantages of Using Arrays

- Need to define a size for array
  - High overestimate (waste of space)
- redundant memory space
  - it is difficult to estimate the size of array

# Linked List

- Series of nodes
  - not adjacent in memory
  - contain the element and a pointer to a node containing its succesor



# Pointer Recap

- Computers store data in memory cells
- Each cell has an *unique address*
- Address-of operator &
- De-reference operator *

| Addr | Content | Addr | Content | Addr | Content | Addr | Content |
|---|---|---|---|---|---|---|---|
| 0 | x: 2 | 1 | y: 6 | 2 | k: 8 | 3 | m: 9 |
| 4 | a[0]: 'a' | 5 | a[1]: 'b' | 6 | a[2]: '\0' | 7 | 3 |
| 8 | 2 | 9 | 3 | 10 | 2 | 11 | 10 |
| 12 | … | 13 | … | 14 | … | 15 | … |

# Linked Lists

- A *linked list* is a series of connected *nodes (cells)*
- Here is a linked list of integers:

  - Data: an integer field
  - Pointer: stores the address of the next node

- The end of the list is marked by a **NULL** symbol, shown by ϕ in the pointer field.

# Linked Lists in C

- We declare a structure to store a node
  - e.g., in C:

```
typedef  struct _node{
     int data;
     struct _node* next;
} node;
```

  - The declaration `struct _node* next` is recursive, or self-referring – i.e.,
    - Attribute `next` is the same type as that being defined.
    - This allows one `node` to point to another `node` – and so on!

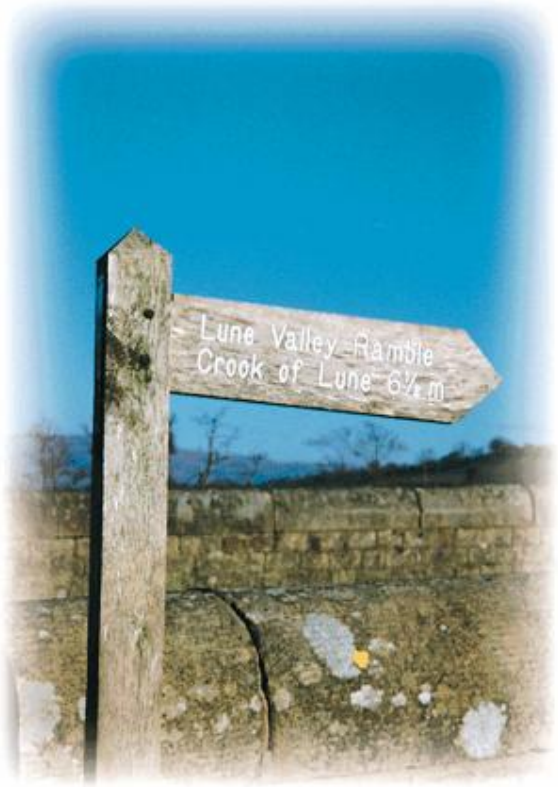
```
typedef  struct _node{ int data;
struct _node* next;
} node;
```

- In C, if we have a struct `node`, with two fields: `data` and `next`

```
node x; // x is an instance of node
node *y; // y is a pointer to a node
```

- To access (e.g.) the data field we say

```
x.data = 12; // dot notation
y = malloc(sizeof(node));
y->data = 12; // arrow notation
```




# THE DYNAMIC NATURE OF LINK LISTS

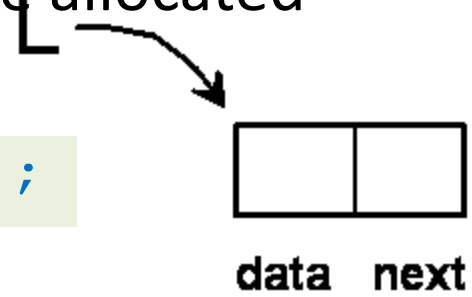
# Advantages of Linked Lists

- Linked Lists are *dynamic*
  - *Dynamic* means they can grow or shrink as required
- Linked lists are non-contiguous
  - Different from arrays
  - It is could be hard to find a large contiguous memory space

# Building a List

- To create memory space for a pointer to point to, in C we use **`malloc`** and **`sizeof`**.
  - **`sizeof`** tells us how many bytes a node structure occupies.
  - **`malloc`** allocates that many bytes from the memory and returns the address of the allocated memory space.

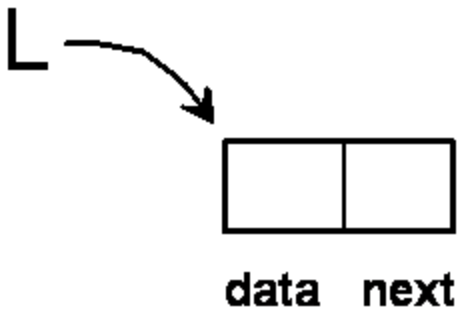
```
node *L = malloc(sizeof(node));
```




  – variable **`L`** points to a newly created **`node`** element

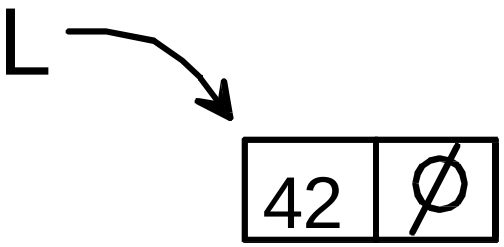
# Building a List (cont)

- C uses arrow notation to assign values, where L is a pointer.




```
L->data = 42;
L->next = NULL;
```

- Now, L points to a one-element linked list:
- Not very interesting!




– How do we grow the list?

# Creating a new node

```
node* new(int _data, node*_next)
{
      node *res = malloc(sizeof(node));
      res->data = _data;
      res->next = _next;
      return res;
}
```

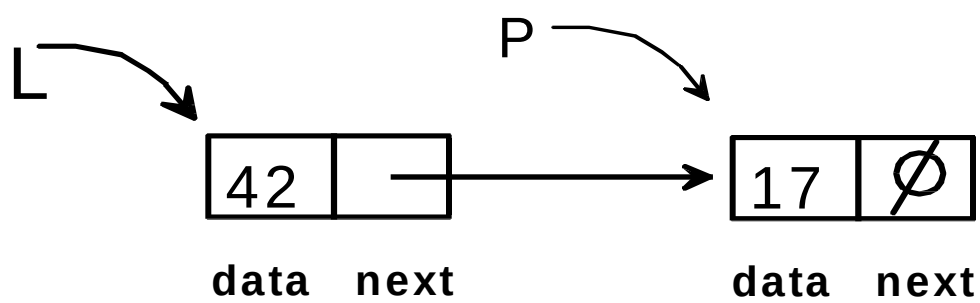
# Building a List

- Let's create two node elements:
  - `node *L = new ( 42, NULL);`
    `node *P = new ( 17, NULL);`

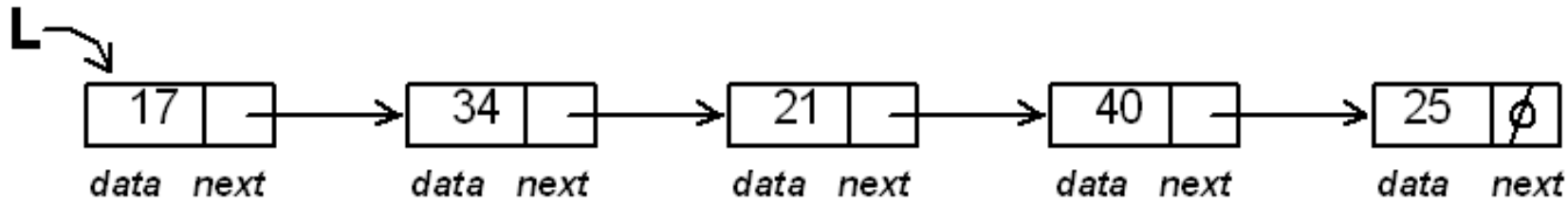
- Now let's just do this ....

  `L->next = P;`

- .... and we get a 2-node list



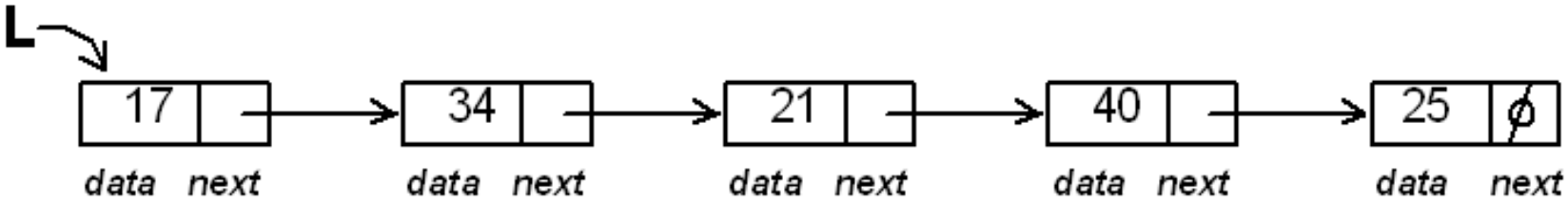
# Scanning Linked Lists




- To access a linked list, we need a pointer to its first node.
  - Lis a pointer-type variable
  - It holds an access pointer to the first node of the list
  - From the first node , we can move to the second and so on.

# Scanning Linked Lists

- Scanning must stop when `NULL` is reached.
- With a linked list, there is no random (or direct) access
  - to reach the 4th node, we must start from the access pointer, and scan along.
  - This is different from arrays (which have random access)

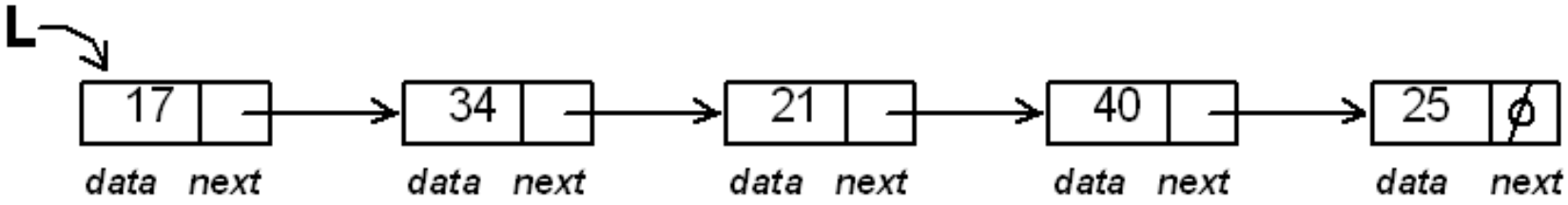


# Scanning

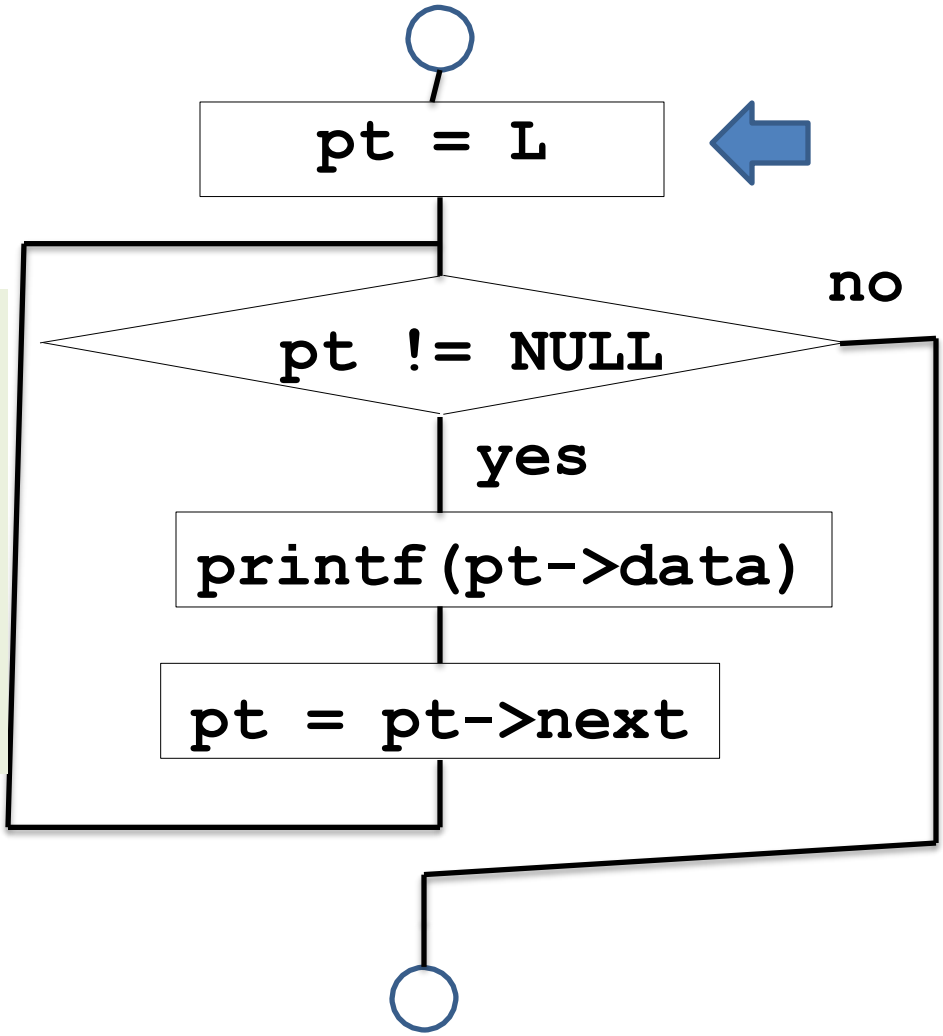
- **Scanning must stop when NULL is reached.**
  - **`pt`** starts at the first node, set by **`pt = L;`**
  - If **`pt!= NULL`**, we haven't reach the last node
  - So we move to the next node, using

    **`pt = pt->next;`**

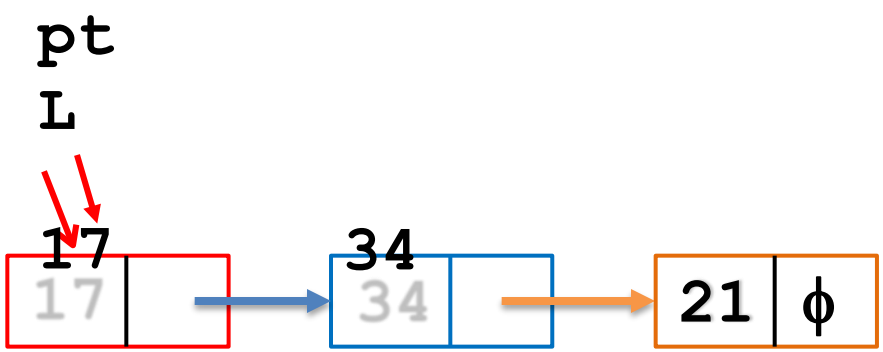
```
void traverse(node *L) {
      node *pt = L;
      while (pt != NULL) {
            printf("%d\n", pt->data);
            pt = pt->next;
      }
}
```



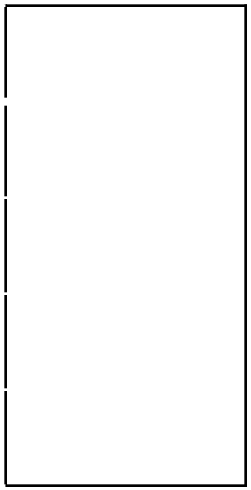
```
void traverse(node *L)
 {
        node *pt = L;
        while (pt != NULL)
        {
        printf("%d\n", pt->data);
        pt = pt->next;
        }
 }
```

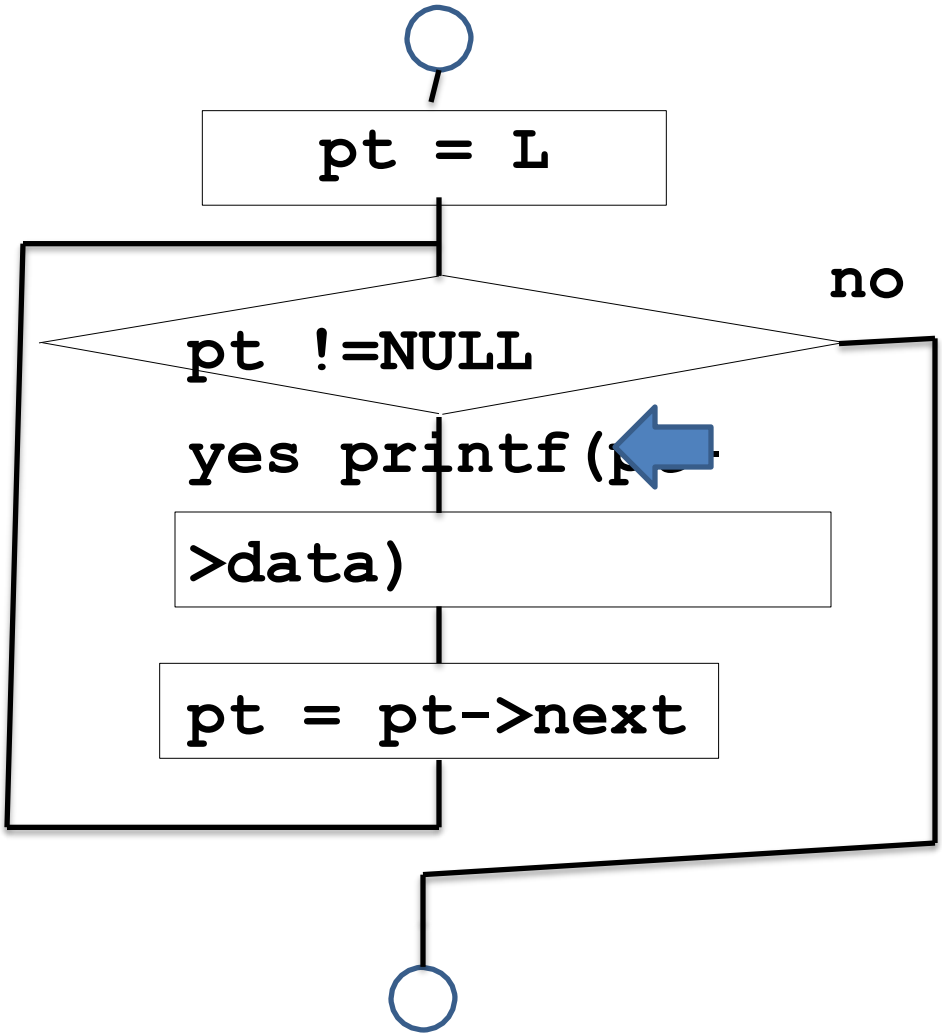


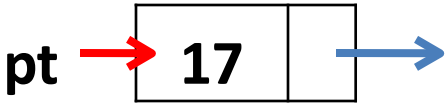

Initialise scan pointer

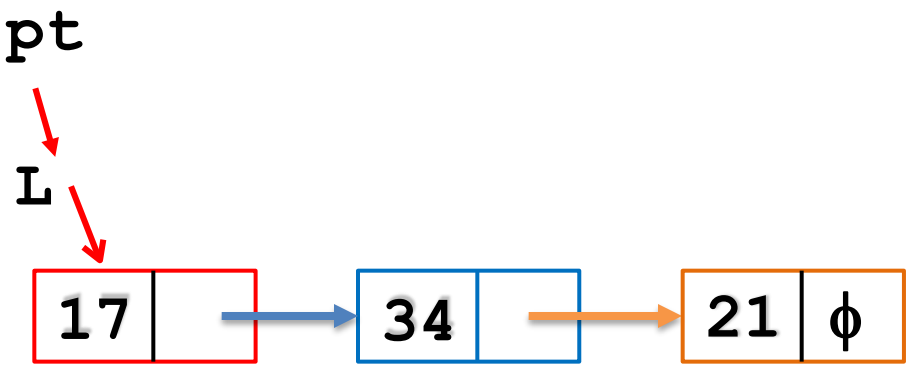


Output



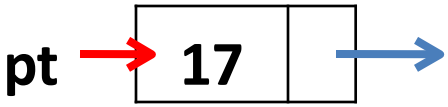


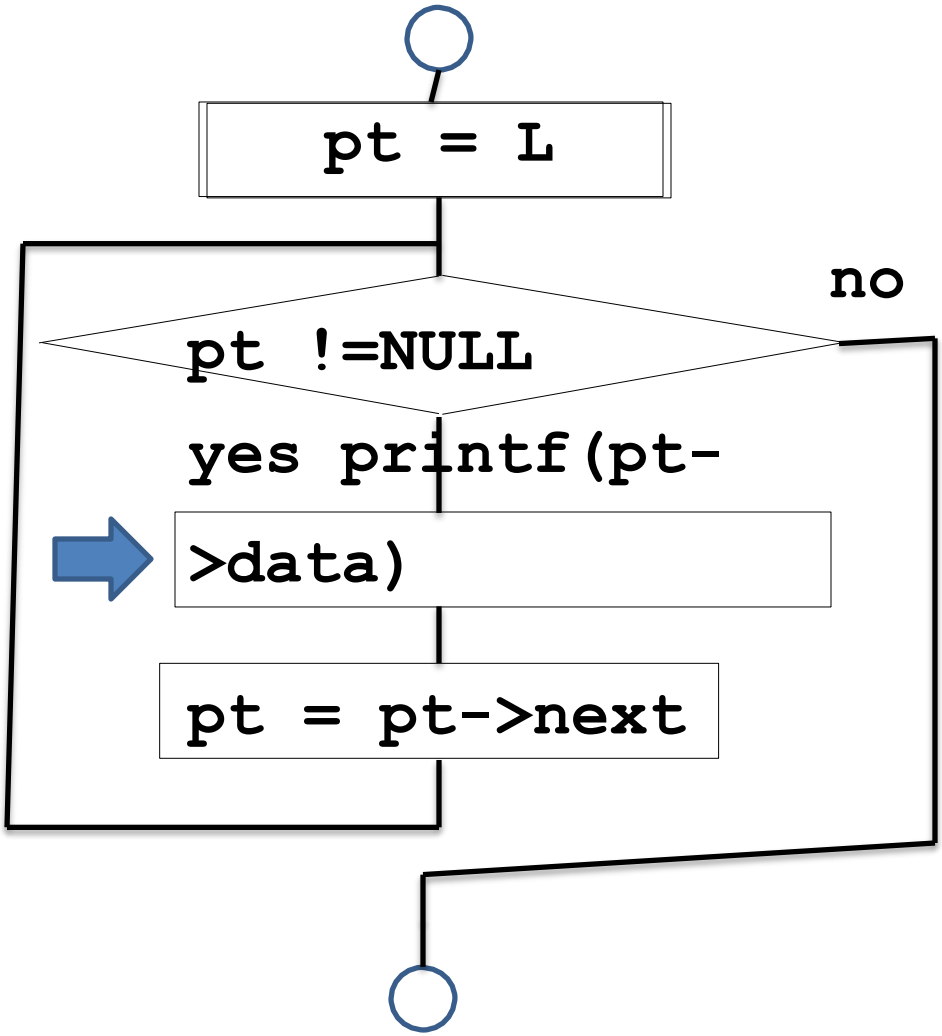



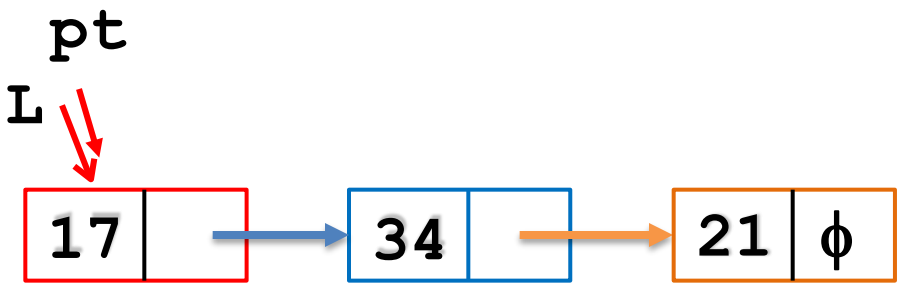



Output

| |
|---|
| 17 |

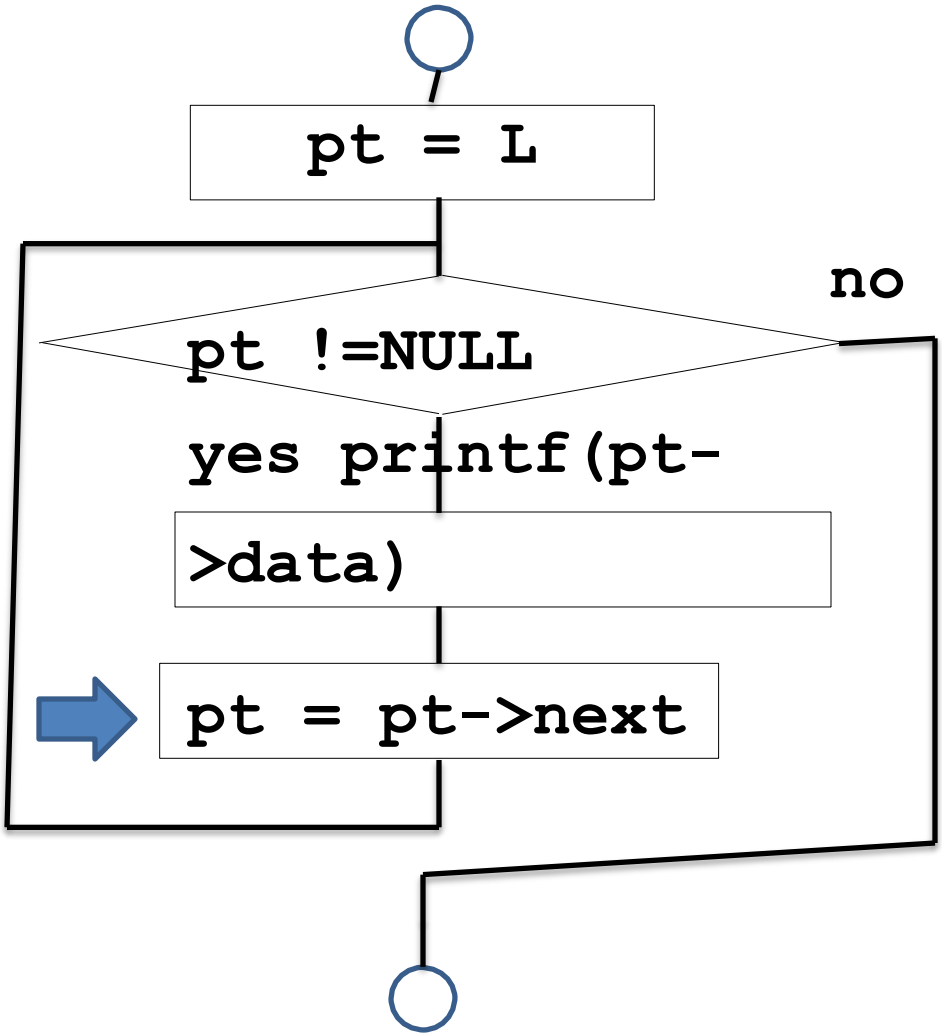


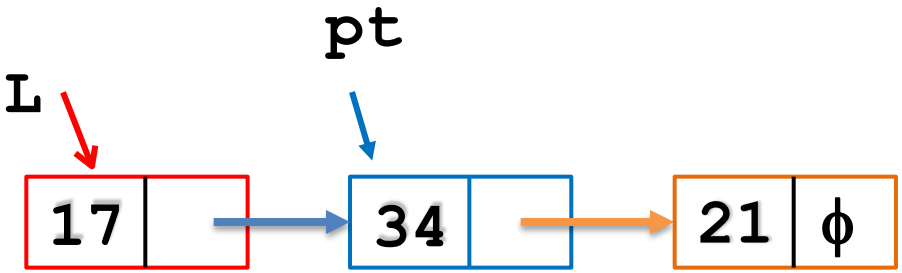



Output

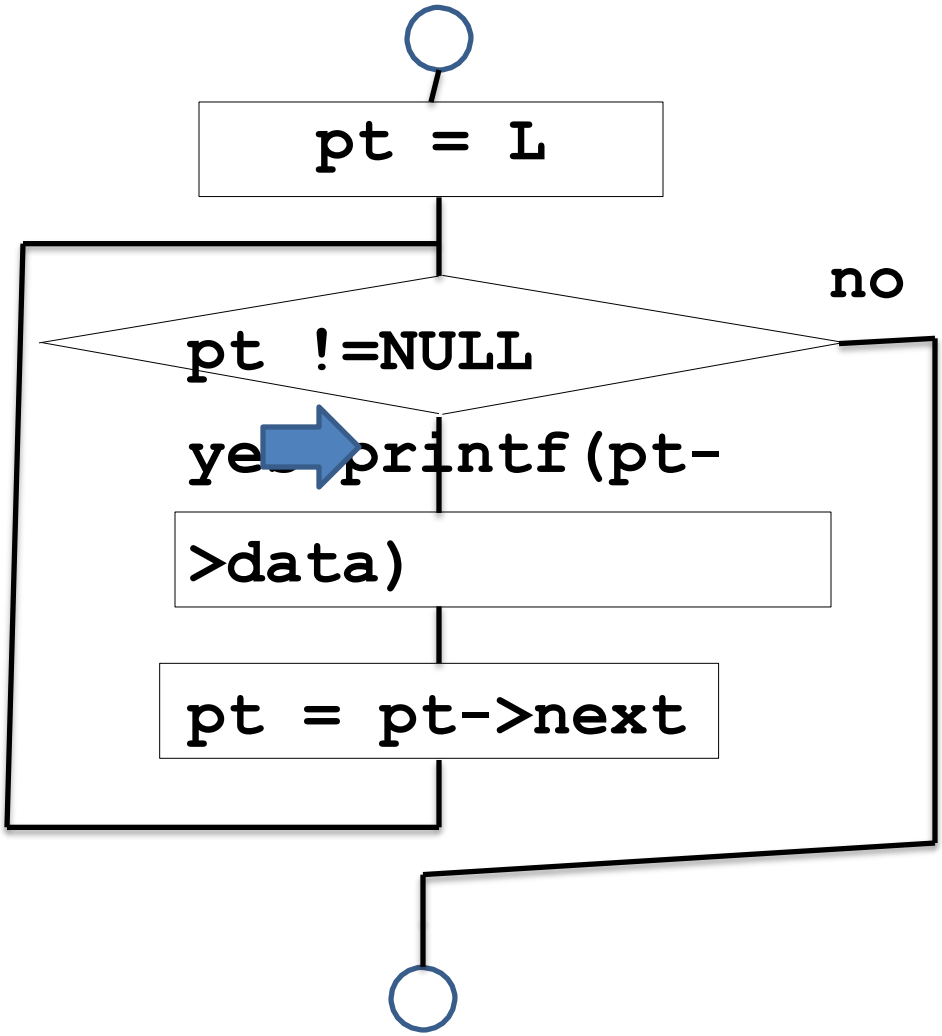
| |
|---|
| 17 |

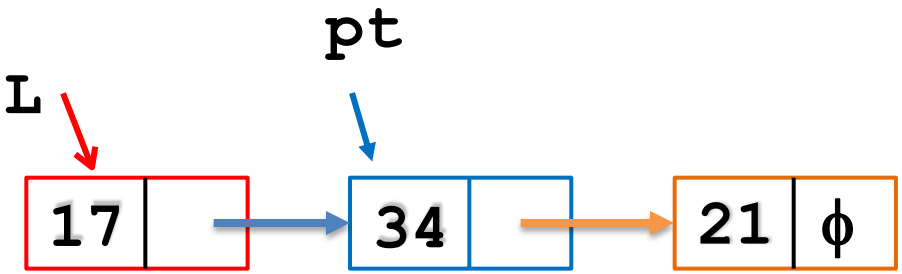




Output

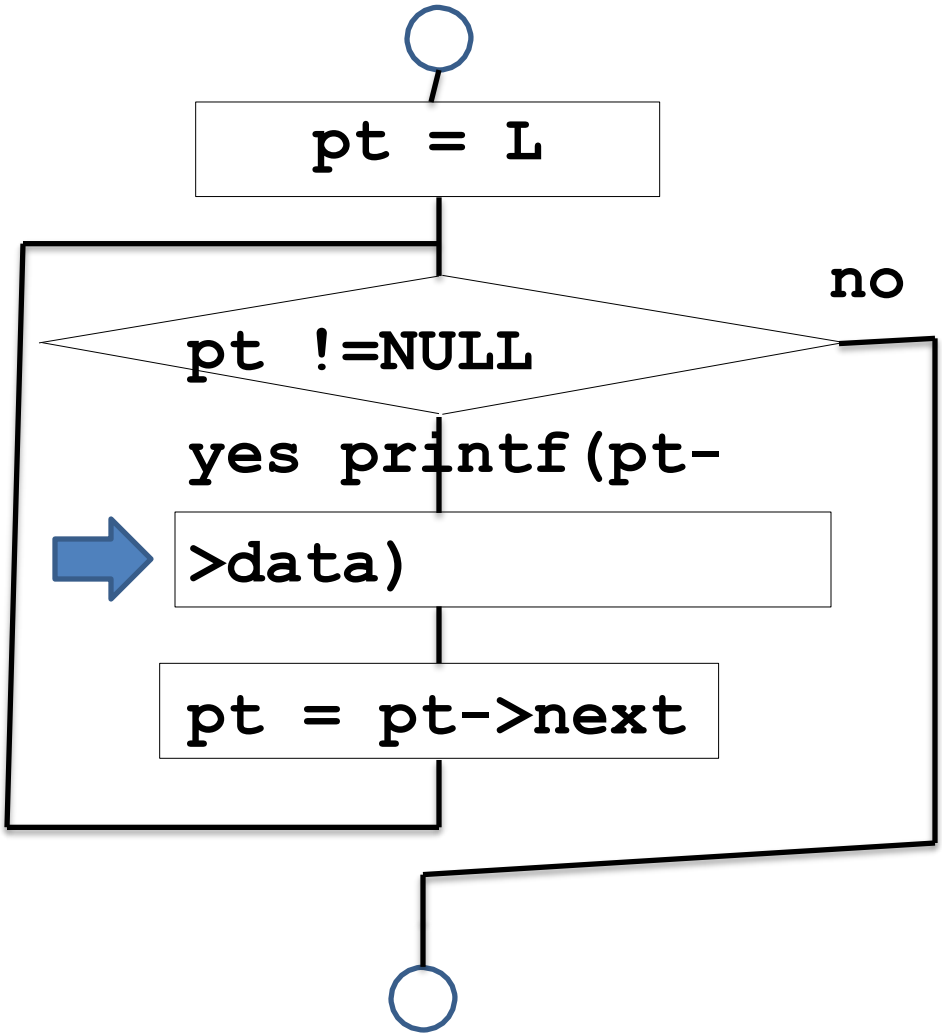
| |
|---|
| 17 |

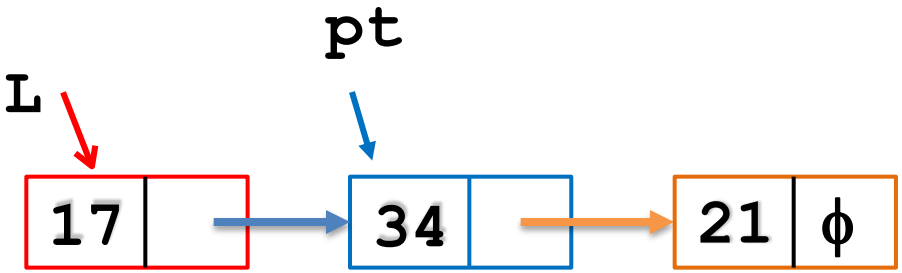


Output

| |
|---|
| 17 |
| 34 |





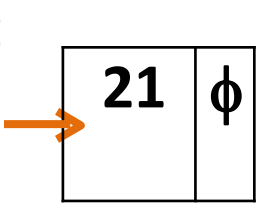
Output

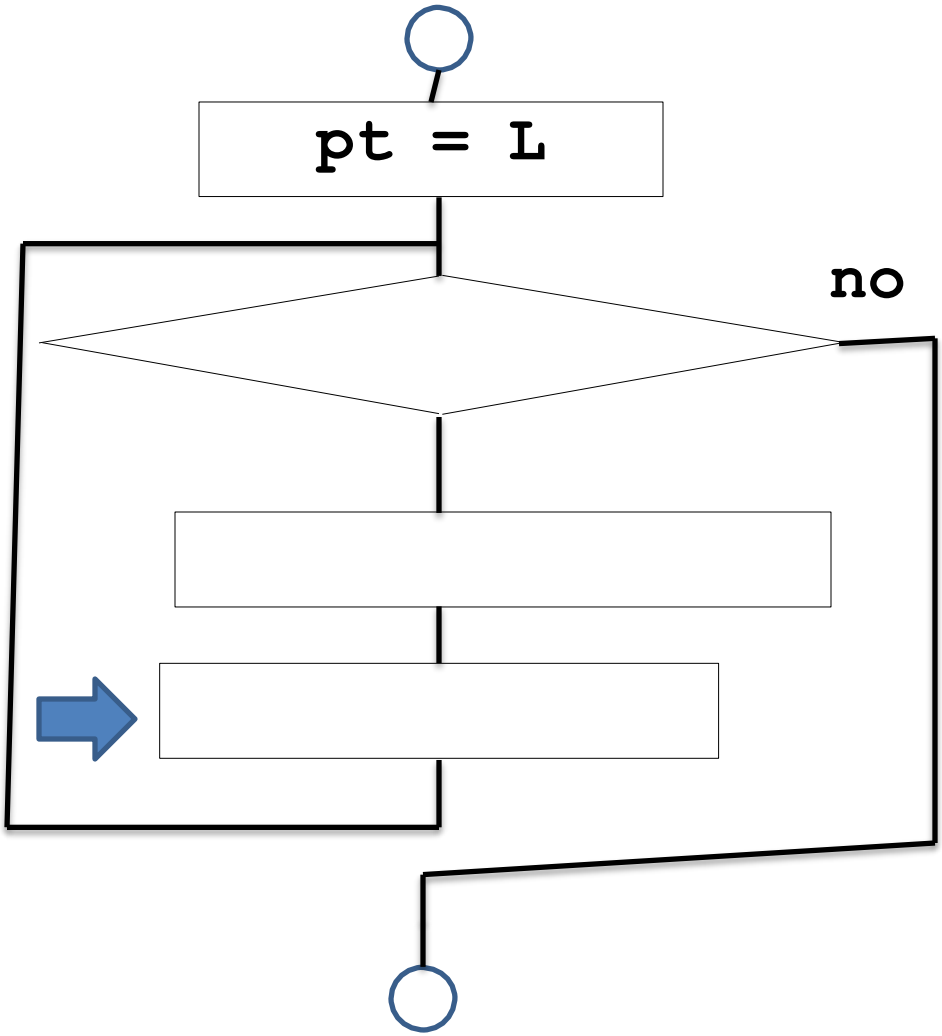
17

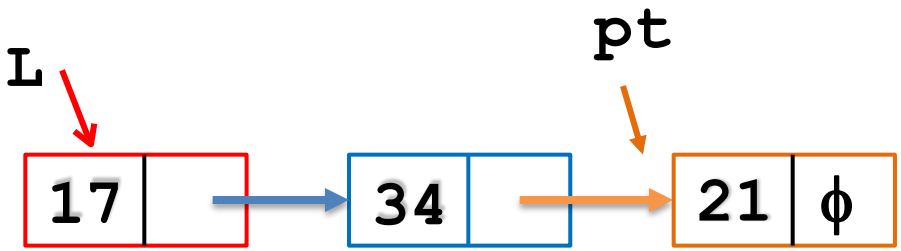
34

```
pt !=NULL
yes printf(pt->data)

pt = pt->next
```

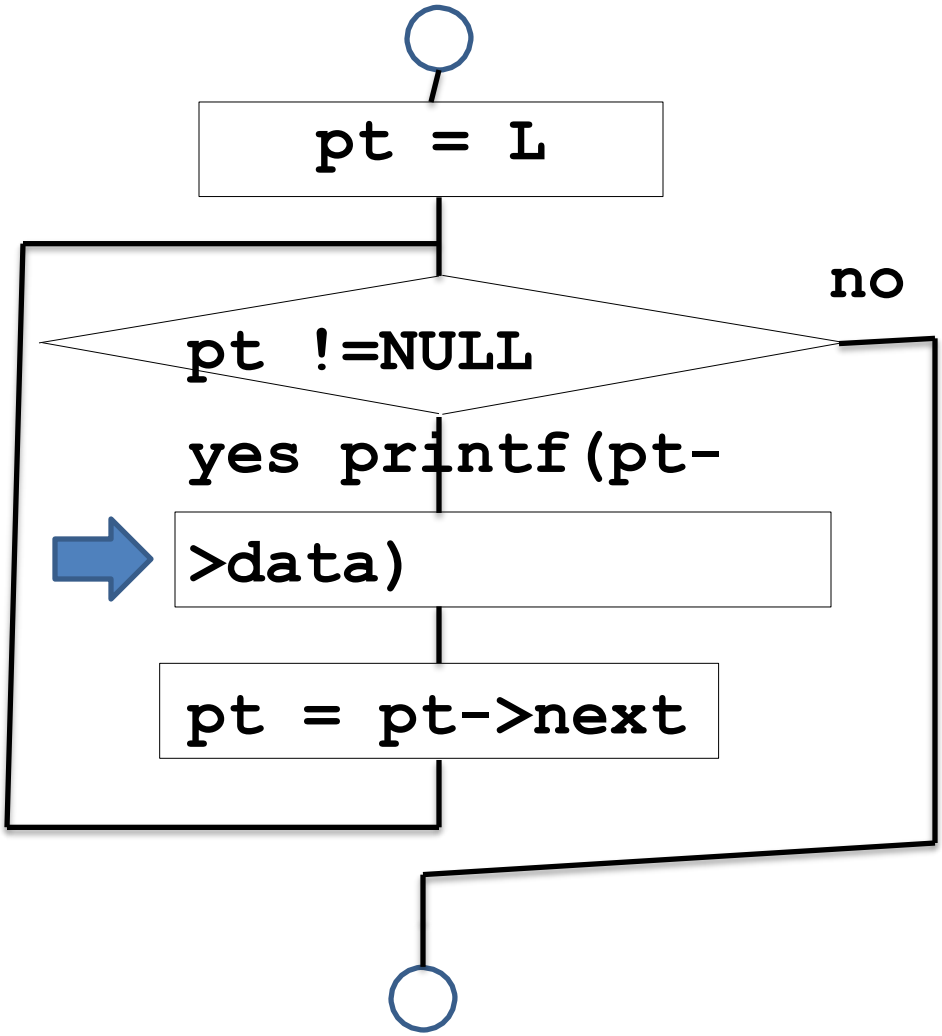


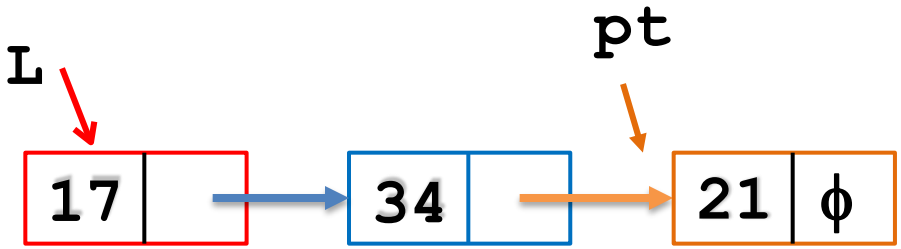



Output

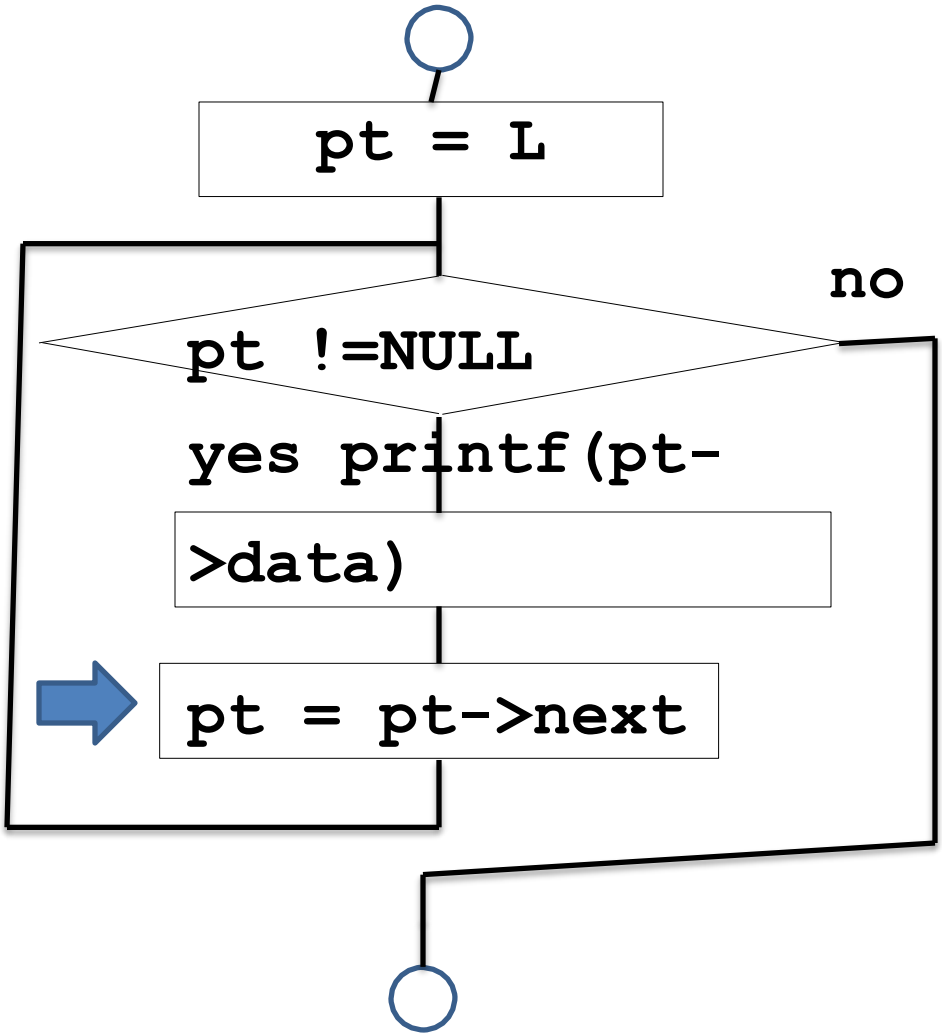
| |
|---|
| 17 |
| 34 |
| 21 |



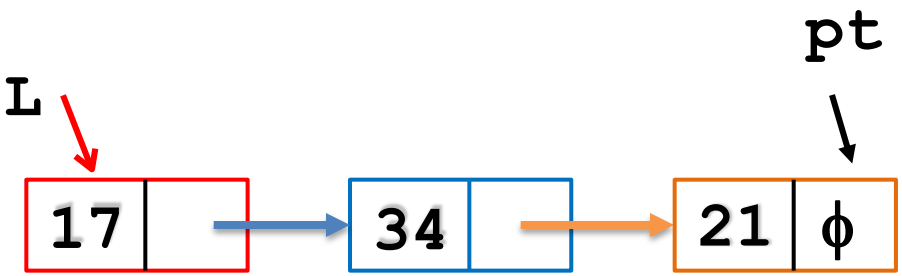


Output

| |
|---|
| 17 |
| 34 |
| 21 |

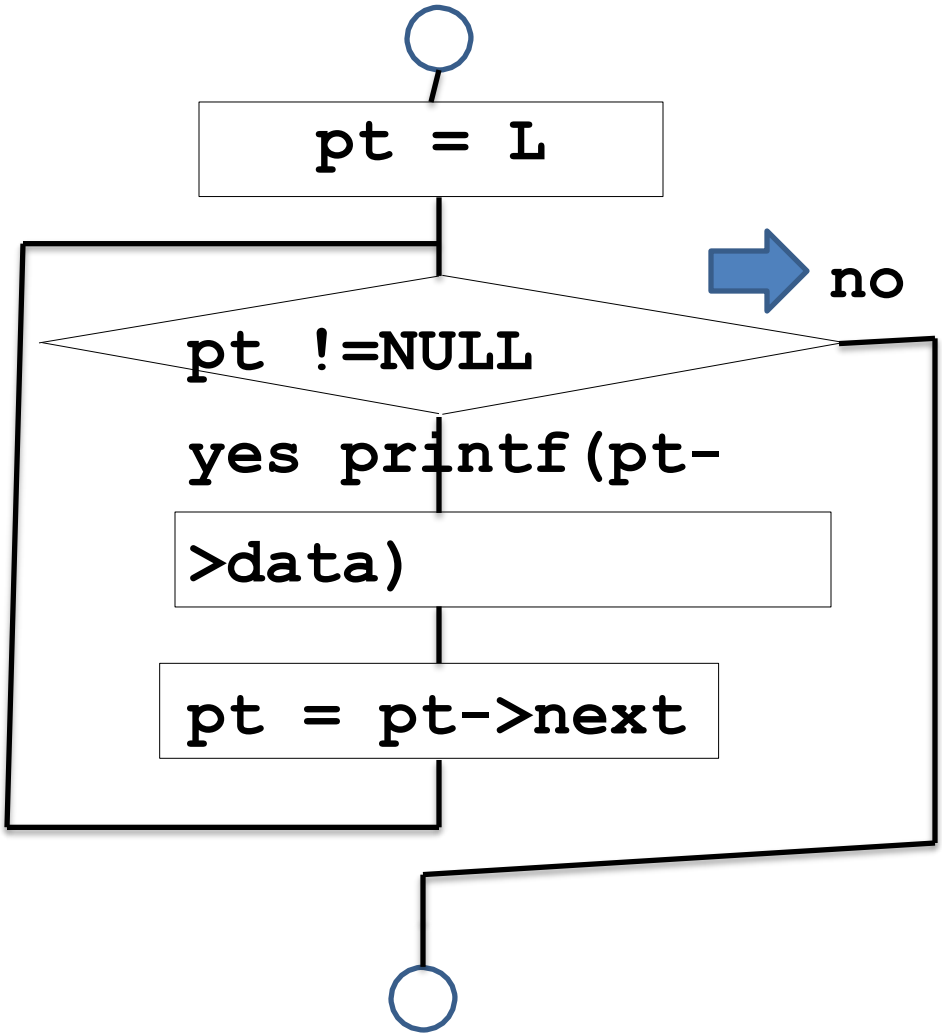



pt → ϕ

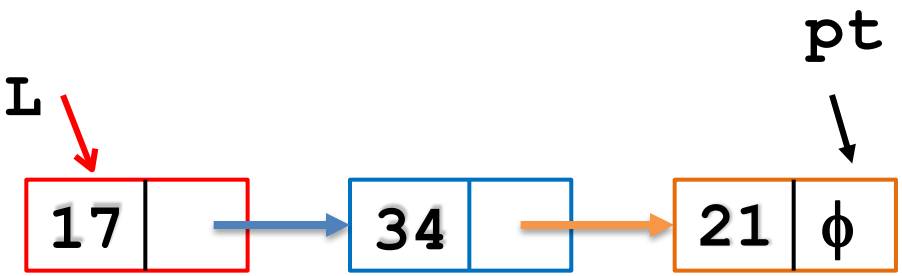


Output

| |
|---|
| 17 |
| 34 |
| 21 |

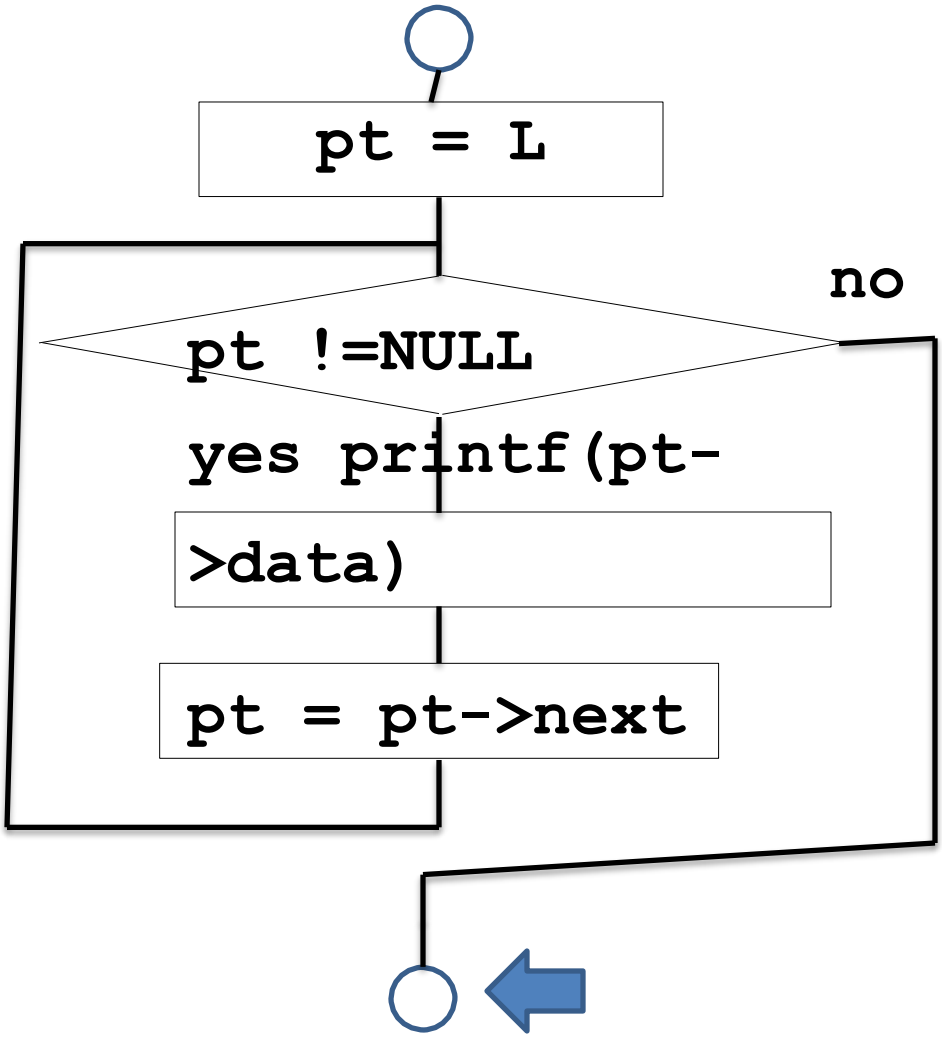



pt → ϕ

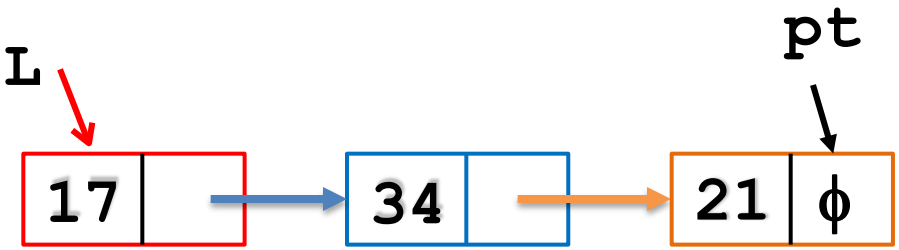


Output

| |
|---|
| 17 |
| 34 |
| 21 |



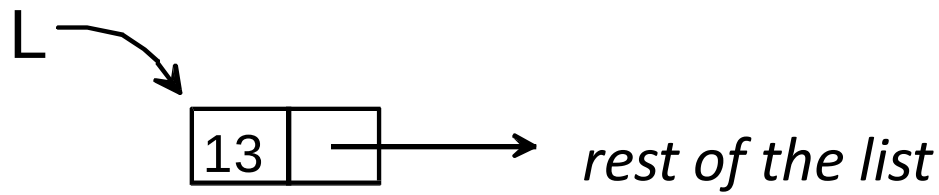

pt → ϕ



# GROWING THE LIST: ADDING AT THE FRONT

# Adding at the Front

Existing list:



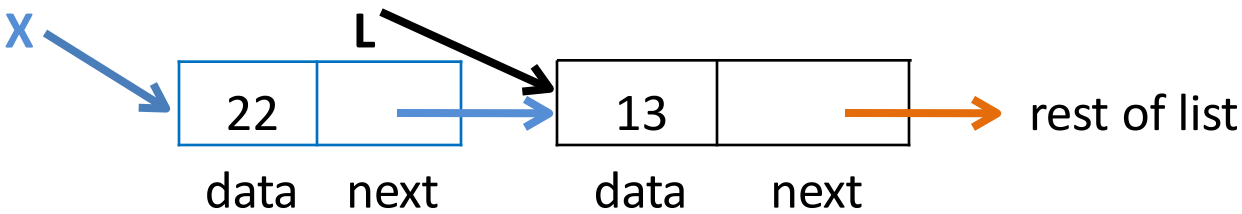

- Given any **non-empty** list L:
  - Let's add 22 – first make a node `X` to hold it:

```
node *X = new (22, NULL);
```

# Adding at the Front

- Now make the next point to the '13' node, L:

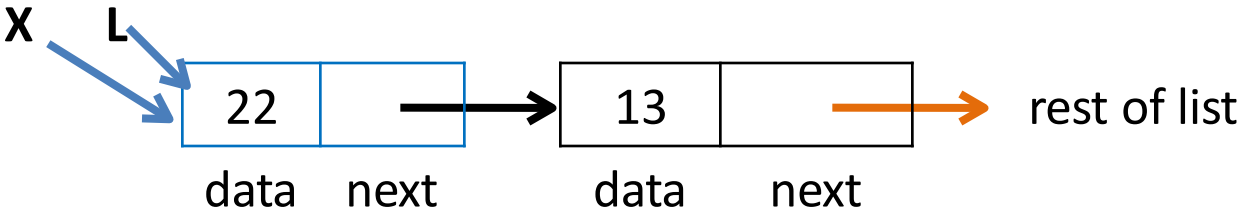
```
X->next = L;
```




- Finally, make L point to the new node, X:

```
L = X;
```

(note that X is no longer needed)

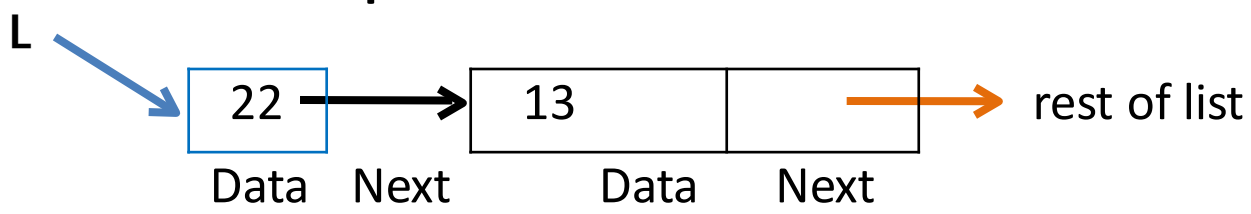


# Adding at the Front

- Using these three steps:

```
node *X = new ( 22, NULL);
X->next = L;
L = X;
```

- we finish up with:



More linked list operations

# ADDING A NODE AT THE END

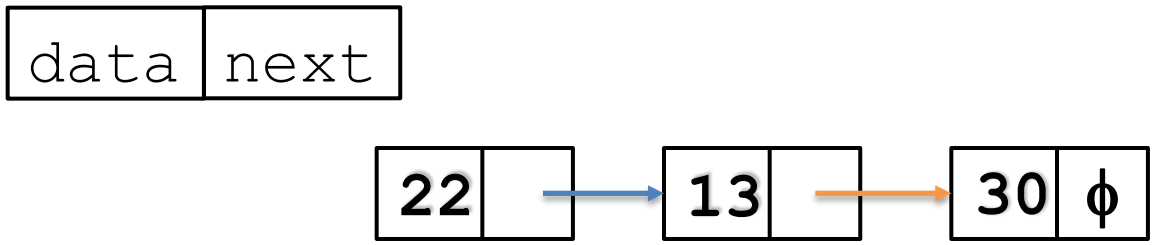
# Linked Lists – Adding at the Back

- *Steps* :
  - `L` points to the *first* node of the list
  - Find the *last* node of the list, `z`
  - Create a new node
  - Make node `z->next` points to the new node

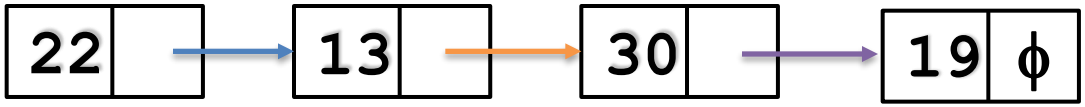
- To find the last node, we must *scan* the list

# Adding a node at the end

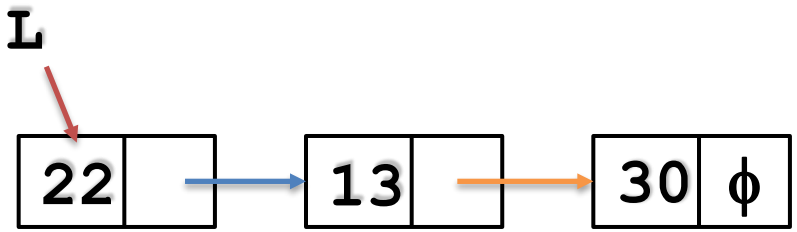
Here is our list at the beginning.

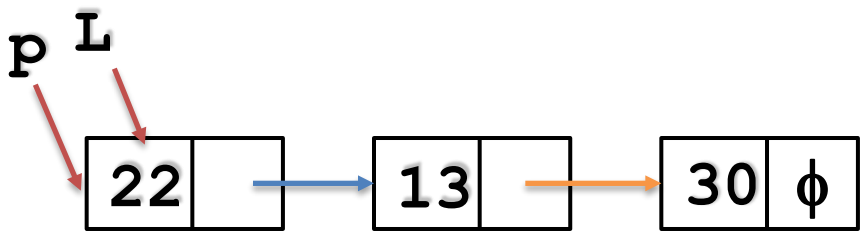



Here is our list after adding a new node containing **the value 19**

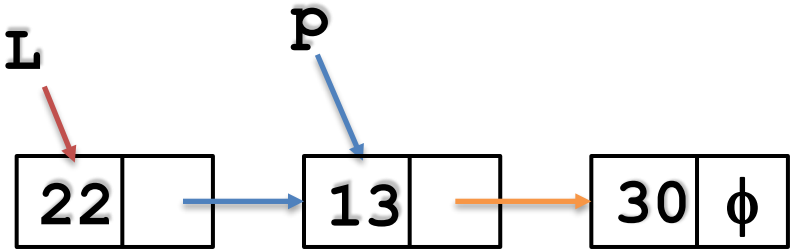


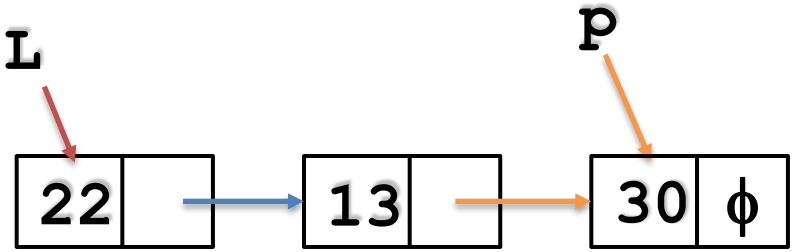
# Adding a node at the end



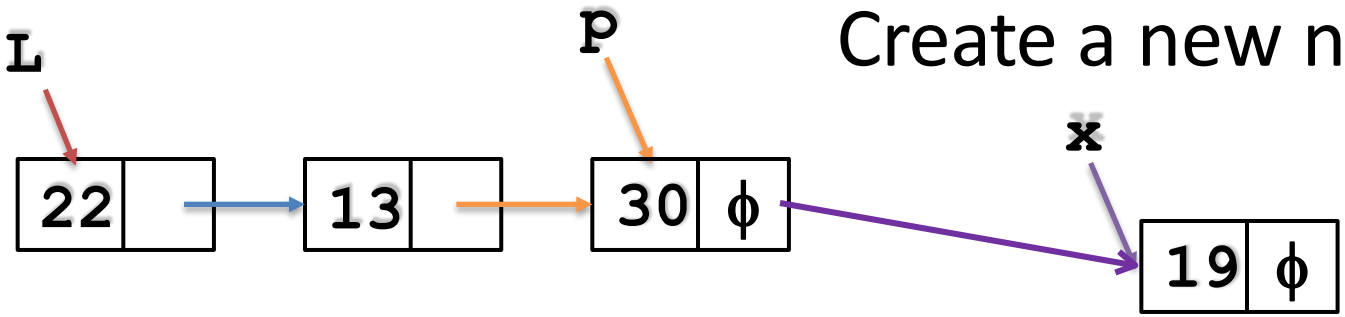




We need to move our "current element pointer" **p** to the last element in the list.



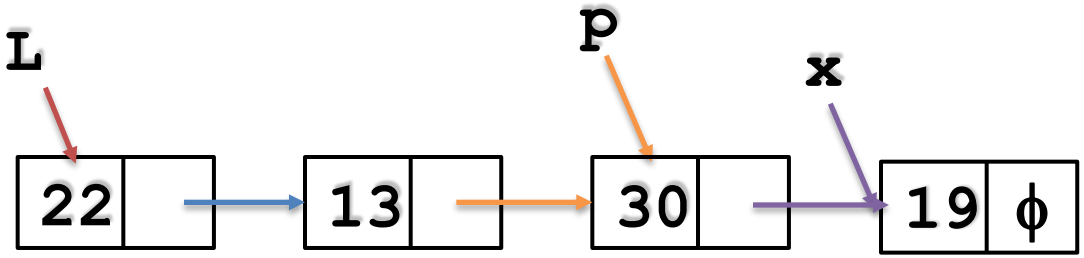



# Adding a the node at end




Create a new node, **x**.

Make **p->next** point to **x**.

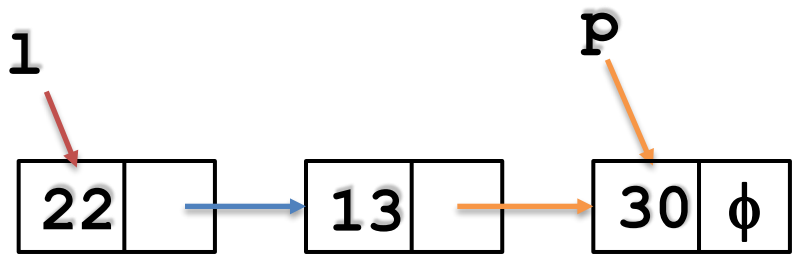


# Scanning to the end of the list

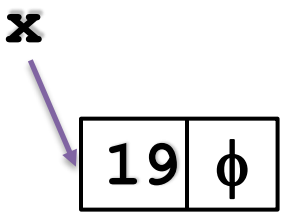
- Scan to the end of the list…

```
p = L; //initialise scan-pointer p
while ( p is not the last node ) //find last node
     p = p->next;
```

- Stops with **p** on last node of L.



# Adding a node at the end




- To add 19 at the end
  - create new node:

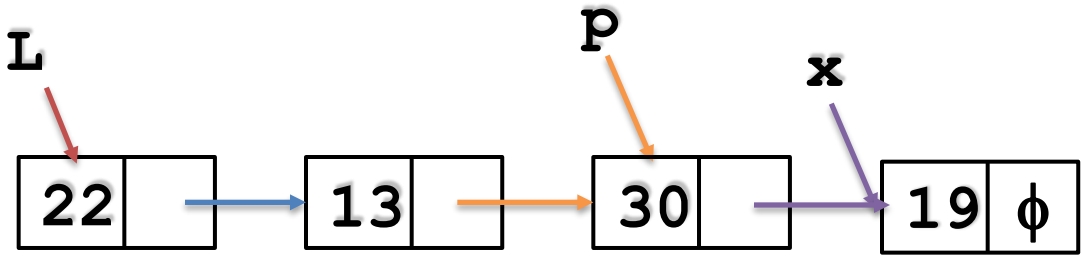
```
node *x = new(19, NULL);
```

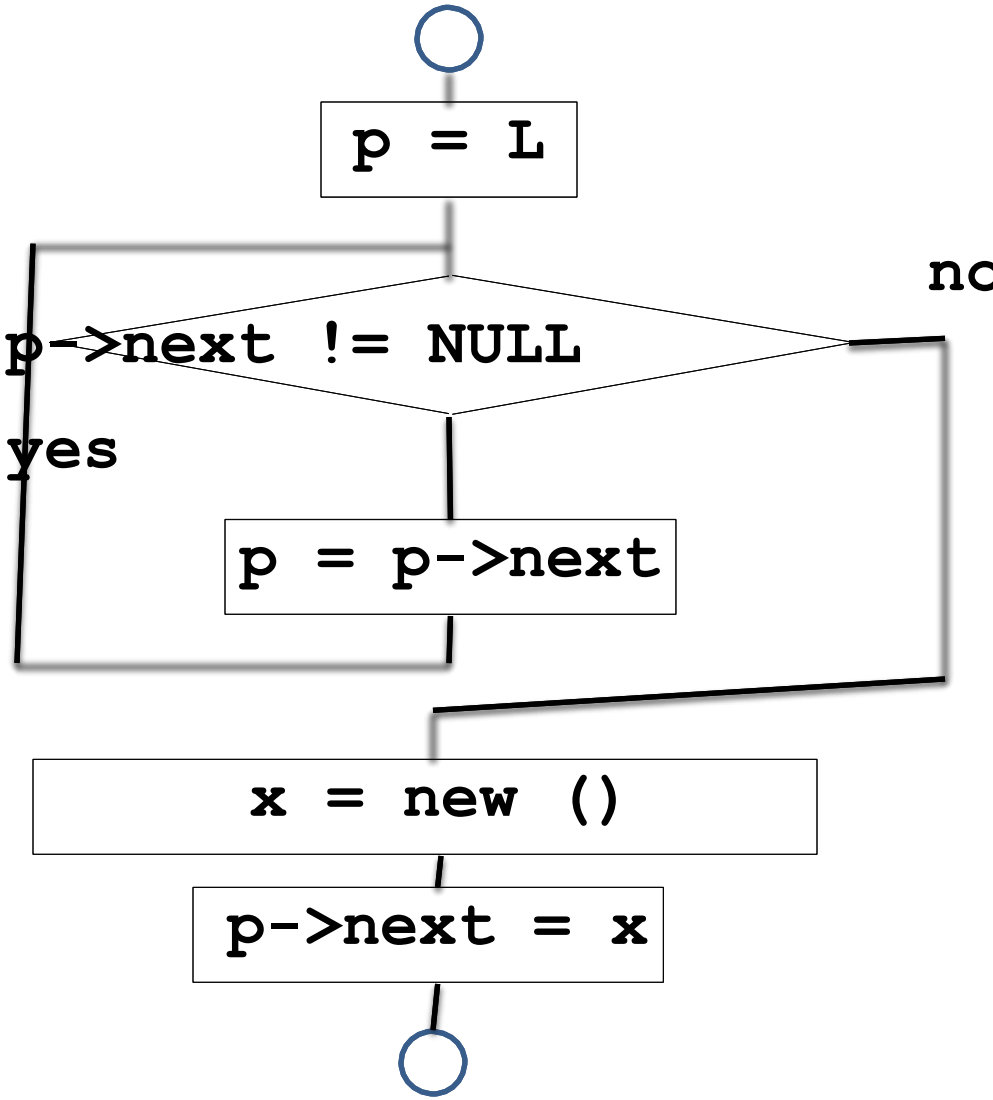
  - and attach it to **p**:

```
p->next = x;
```

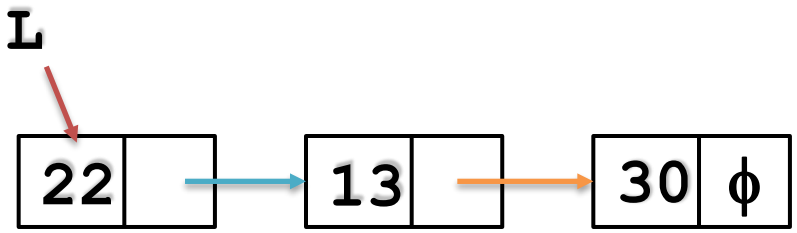
- or, we combine the last two operations:

```
p->next = new (19, NULL);
```






```
p = L;
while (p->next!= NULL)
        p = p->next;
node *x = new (19, NULL);
p->next = x;
```



# Adding node at End : The Whole Business

```
p =L; //initialise scan-pointer    p
while (p->next != NULL)//find last cell
   p = p->next;
p->next  = new ( 19,    NULL);
```

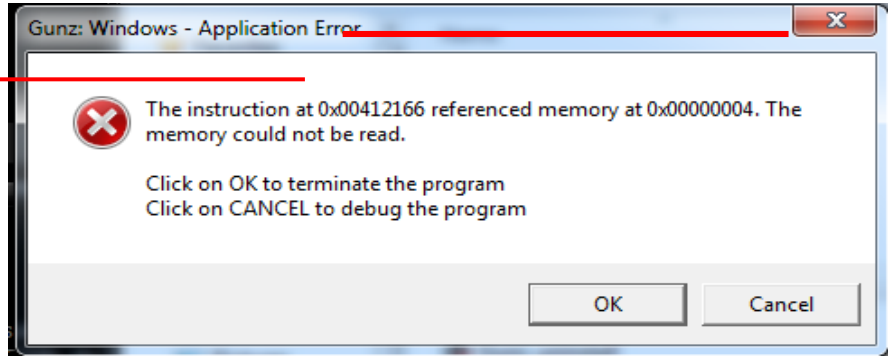
Quiz:

What would happen if `L` was NULL? How should we adapt the code to cope?

# Adding Cell at End : The Whole Business

- If `L` is NULL ….
- This generates a run-time error
  - Because there is no `p->next`

```
p =  L;    //    initialise scan-pointer p
  while (p->next!= NULL) //    find last cell
p = p->next; p->next= …;
```



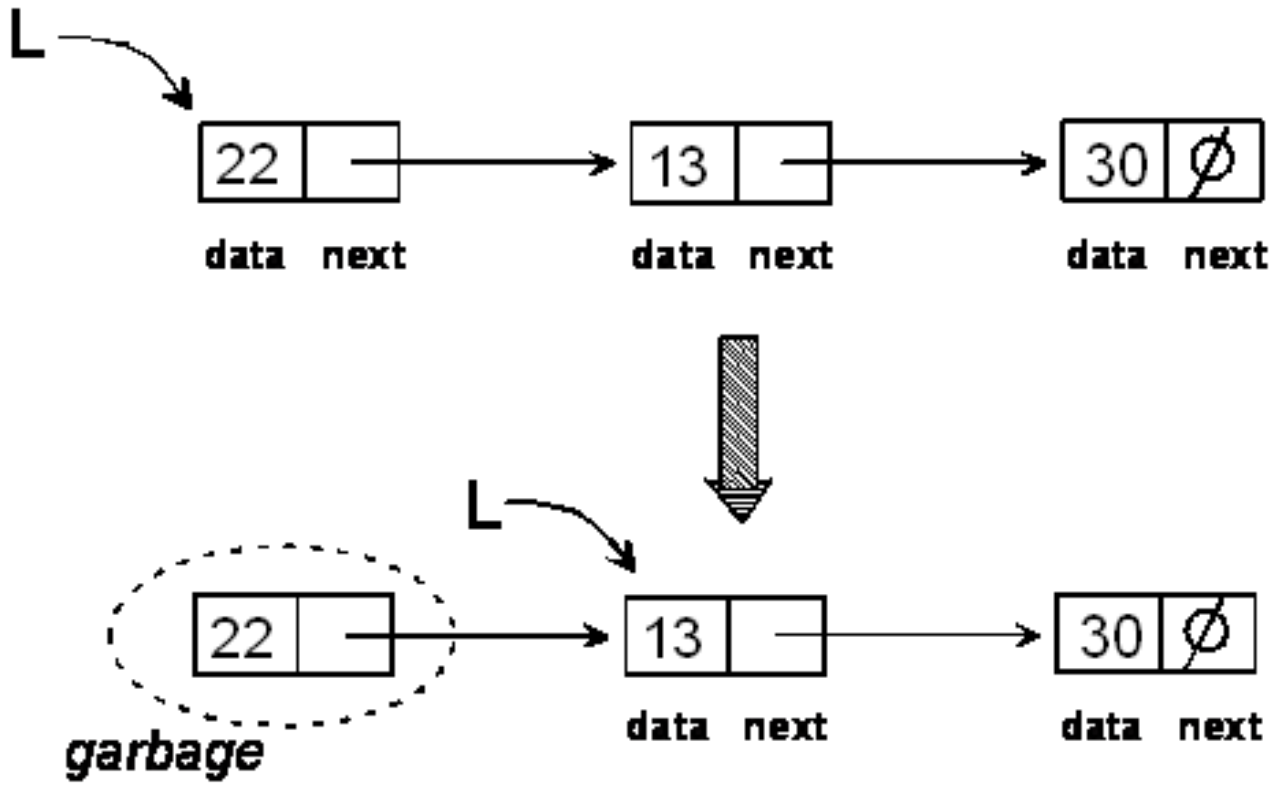
# Adding Node at End : The Whole Business

```
if (L != NULL)
{
     p = L;
     while (p->next != NULL)
          p = p->next;
     p->next = new ( 19, NULL );
}
else {
     L = new ( 19, NULL);
}
```

# REMOVING THE FIRST NODE

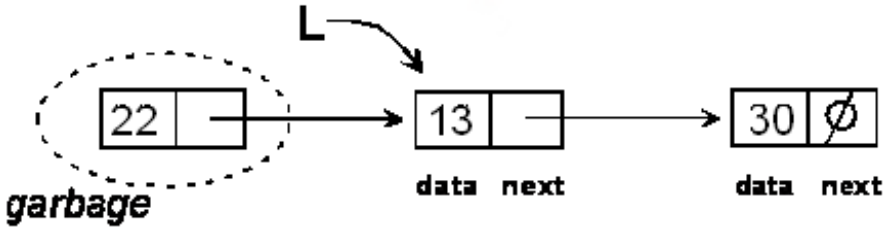
# Removing the First Item

```
int x = L->data; // keep data value
L = L->next;     // delete first cell
return x;        // return result
```

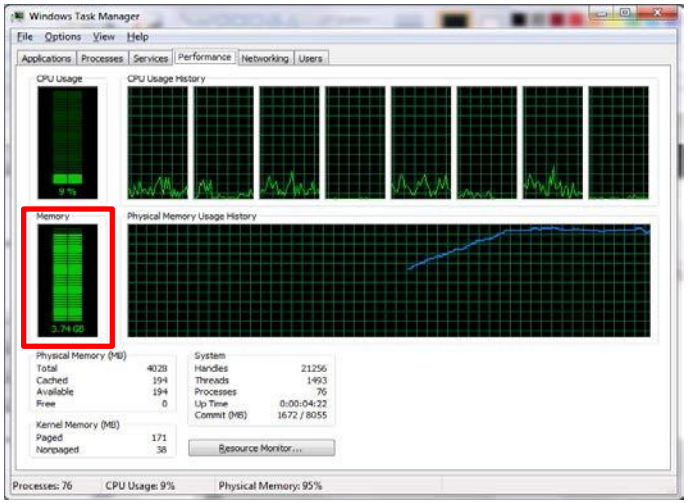



Afterwards, X holds the value 22.
The “garbage” cell can be re-used later.

# Why is it “garbage”?




- It’s garbage because
  - Nothing is pointing at it anymore, and therefore
  - it can no longer be accessed (or used) by anyone
- It’s just taking up space …
  - that can be reused later …

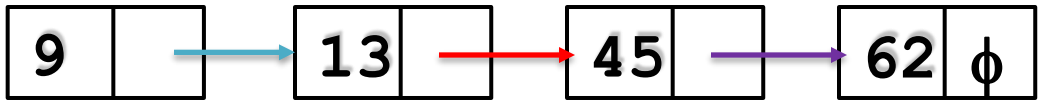


# DELETE THE MIDDLE NODE FROM A LIST

# Deleting node in middle

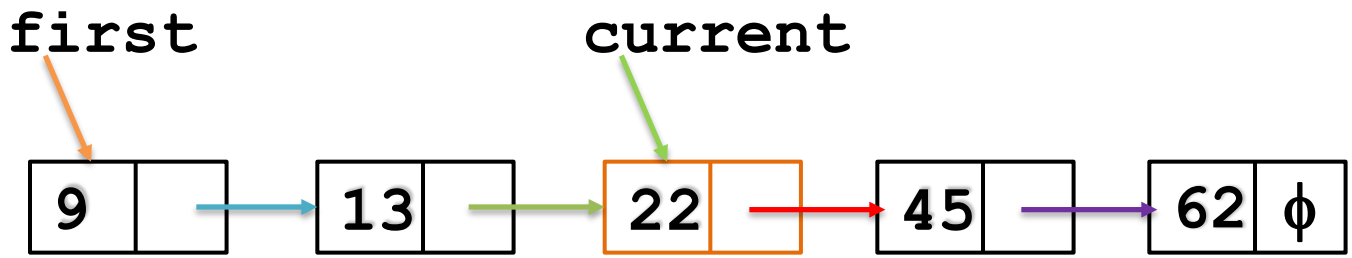
Here is our list at the beginning.

Here is our list after deletion.

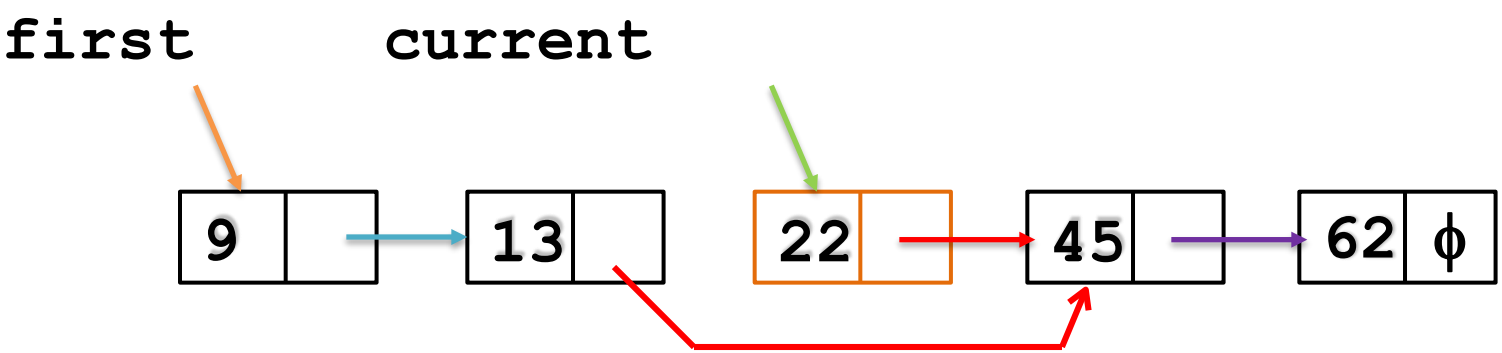


# Deleting node in middle

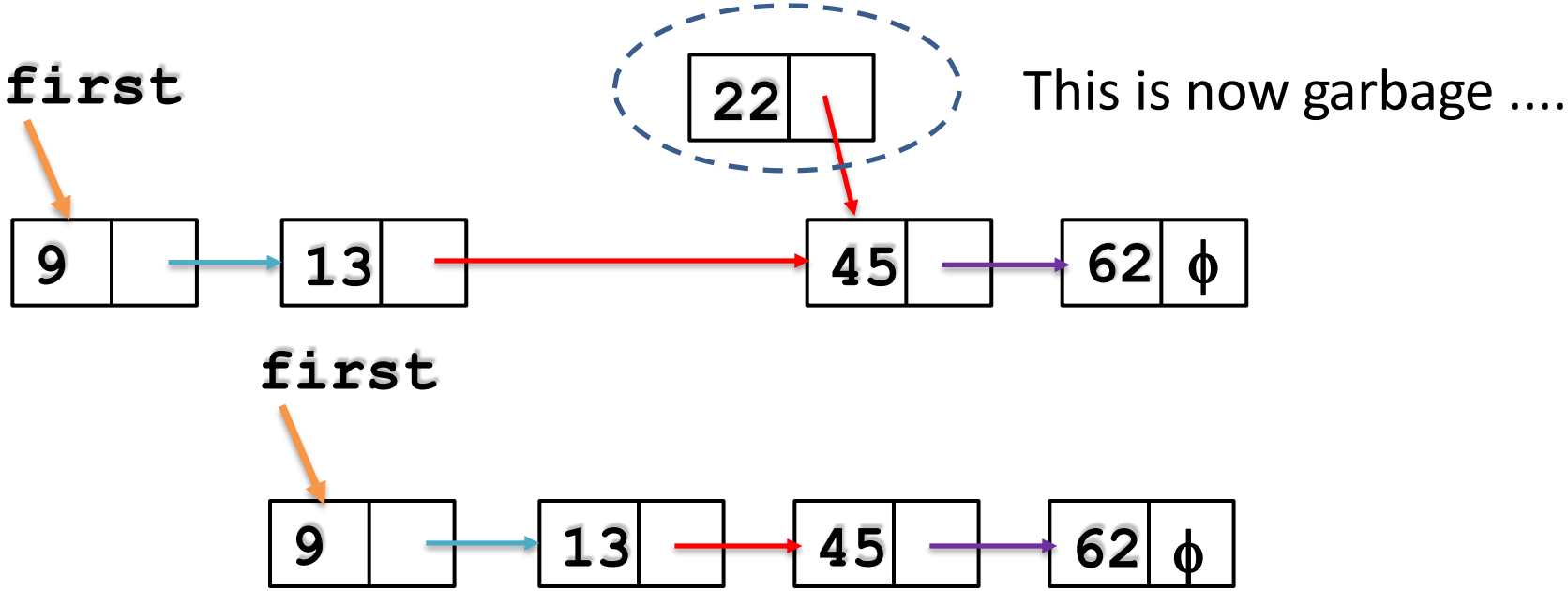
We need to find the node to be deleted.




We need to make the next pointer of the previous-to-current node point to the same thing as the `current->next` pointer.

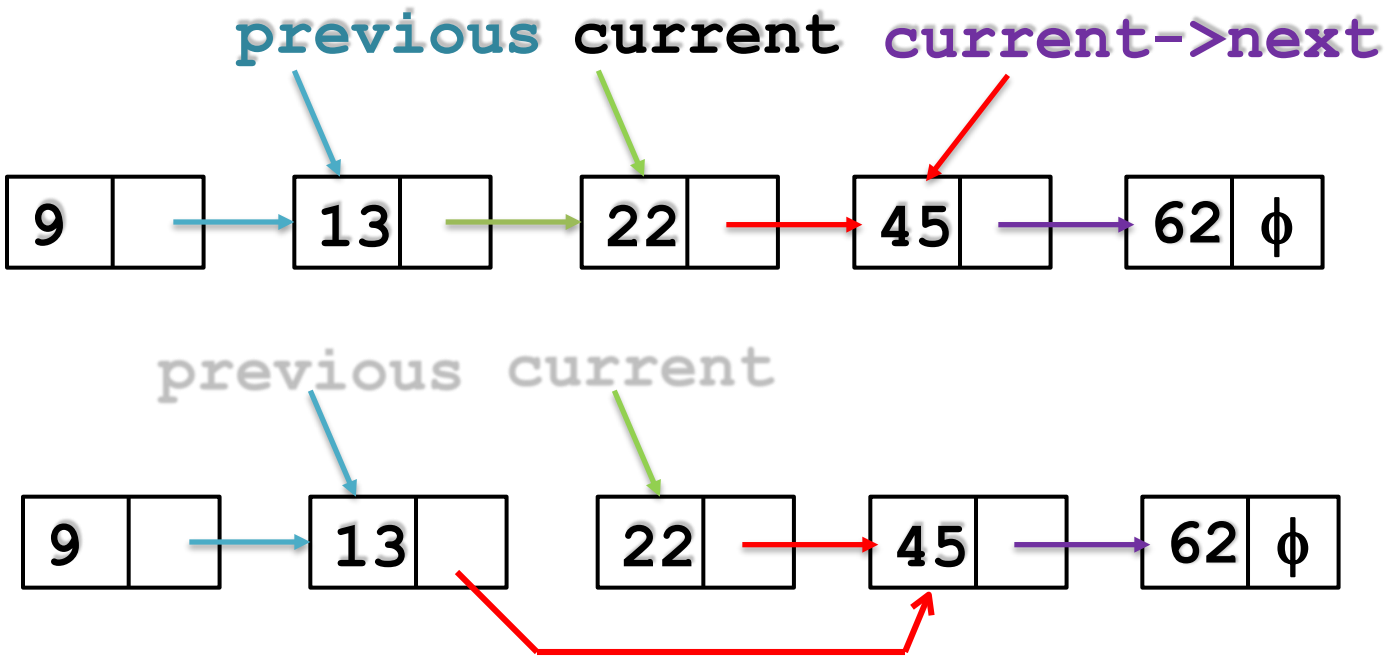


# Deleting node in middle



# Algorithm

- For our algorithm, not only do we need a **current** pointer, we need a "**previous**-to-current" pointer.
- **previous->next = current->next;**
  **current = NULL;**



# Delete middle node from List

```
bool found = false;
node *current = first;
node *previous = NULL;

while (current != NULL)
{
if (current->value == x) {
        found = true;
        break;
}
else {
previous = current; current = current->next;
}
}
if (found) {
        if (previous != NULL)
                previous->next = current->next;
        else
                first = current->next;
}
```

# DOUBLY LINKED LISTS

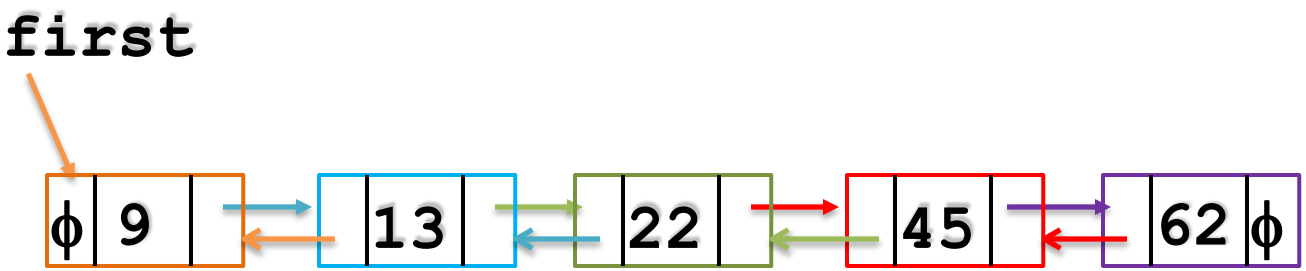
# Multiple pointers

- If we can have one pointer (going forward) why can't we have another pointer (going backward)?

```
typedef struct _node {
  struct _node *previous;
  int data;
  struct _node *next;
} doubleLL;
```

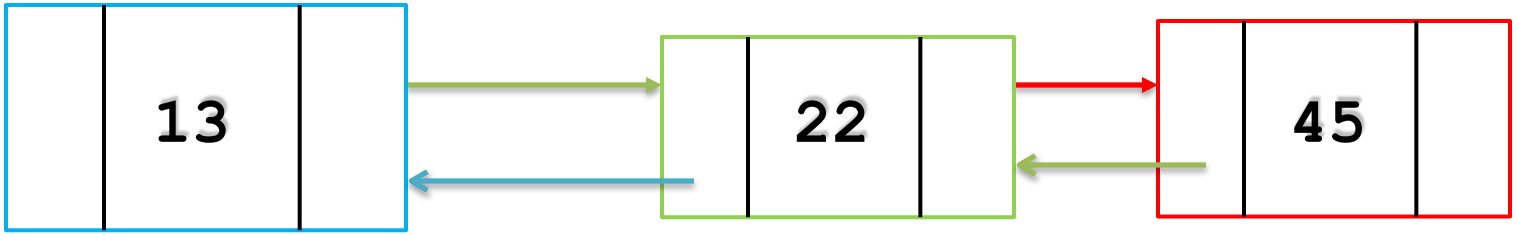
The structure of a doubly-linked node

| previous | value | next |
|---|---|---|



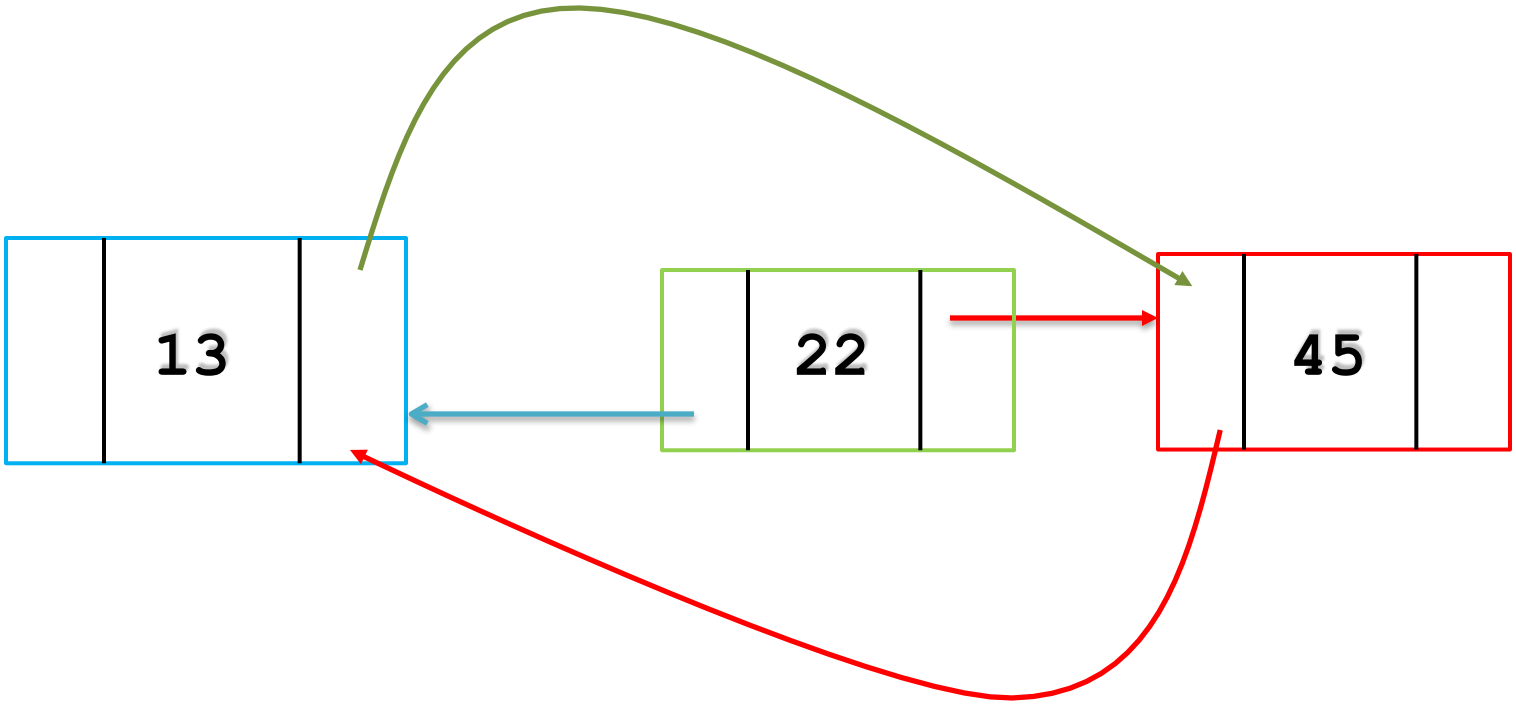

An example of a doubly linked list

# Deleting a node from the middle



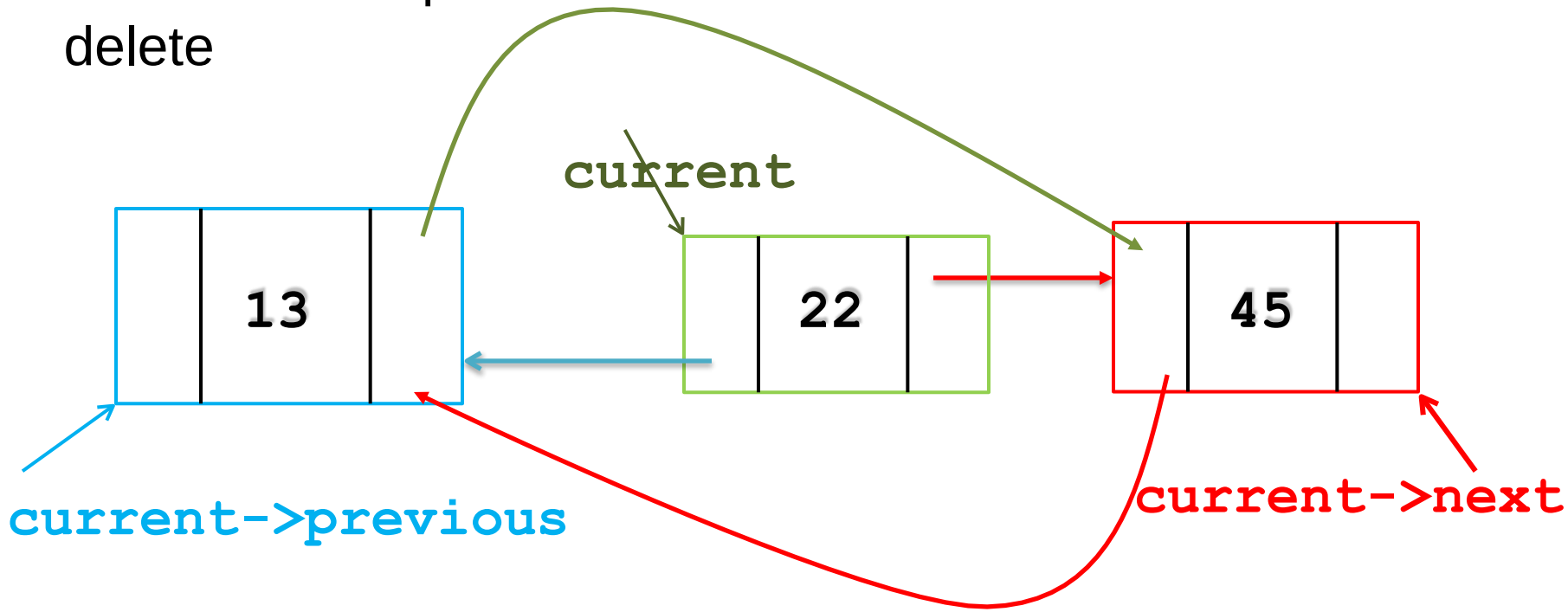

We want to delete 22 from the doubly linked list

# Deleting a node from the middle



# Deleting a node from the middle

Let `current` points to the node we want to delete




```
1.  (current->previous)->next = current->next;

2.  (current->next)->previous = current->previous;
```

```
doubleChainLL* deleteMiddle
            (node *first, int x)
{
    bool found = false; node
    *current = first;

    while (current!=NULL){
    if (current->data == x) {
    found = true; break;

      }
      else current = current->next;
    }

    if (found){
    current->previous->next = current->next;
    current->next->previous = current->previous;
    free(current);
    }
    return first;
}
```

Locating a node that contains value x

Update the doubly linked list

# Exercise

a) Work through the example of the previous slide.
b) Implement them in C.

The only way to master linked list and pointers

c) In our "`deleteMiddle`" function, we haven't taken account of the special cases.

i)what if the list is empty?

ii)what if the node to be deleted is at the start of the list?

iii)what if the node to be deleted is at the end of the list?

Add code to address these issues in C.

# THE END